

6CS007 Project and Professionalism

Final Year Project

Final Report

TruthLens: Fake News Detection Platform

Student Name: Rigen Manandhar

Student ID: 2408621

Supervisor: Mrs. Prakriti Regmi

Reader: Mr. Siman Giri

Submission Date: March 6th, 2026

Declaration

Declaration: I confirm that this report is my own work and that all sources of information have been cited appropriately. No part of this work has been submitted for any other academic award. The work presented herein represents the author's own effort except where due acknowledgement is made.

Student Name: Rigen Manandhar

Student ID: 2408621

Signature: _____

Date: March 6th, 2026

Abstract

There has been an urgent practical requirement for real-time reliable verification tools because of the quick dissemination of inaccurate and misleading (i.e., fake/false) news content over digital venues. This project proposes TruthLens, a text-focused hybrid fake news detection platform developed to assist users who need to evaluate the veracity of English-language news sources. TruthLens will accept a range of input formats to identify and retrieve the relevant news article. All input formats will utilize a combination of (a) providing a headline, claim, or full-text article to verify through direct text entry; (b) extracting a URL-based article from the internet (and providing the URL) for evaluation; and will return a structured credibility assessment, which will include a verdict, risk level, step-by-step analysis trace, and optionally, explainable AI (XAI) outputs generated using LIME (Local Interpretable Model-agnostic Explanations).

The technology behind TruthLens consists of three different levels called layers which work together to identify the credibility of information. The first layer checks the integrity of an information source against an established list of credible sources. Once this step is completed, the credibility of the content is determined by two different models: Model A is a RoBERTa based neural network that was trained using LoRA (Low-Rank Adaptation) to identify truthful claims based on a set of LIAR denoted short claim data sets and other trustworthy news article headlines, while Model B, another RoBERTa network but with different adapters than those of Model A, uses LIAR training data plus additional long-form data to classify long-form articles. Each neural network is accessed via FastAPI running on a server and has a user interface built using Next.js, React, and TailwindCSS. In addition, users can initiate the same verification process through an in-browser Chrome Extension (Manifest V3).

Batch evaluation on curated test sets yielded 94.06% accuracy ($F1 = 94.03\%$) for Model A on headlines and 97.50% accuracy ($F1 = 97.04\%$) for Model B on full articles. The project documents the complete model training journey, including challenges such as shortcut learning on the ISOT dataset, data leakage involving 931 overlapping train/test articles, and iterative solutions including data cleaning, regularization tuning, and multi-source dataset diversification. The system incorporates explicit uncertainty handling low-confidence predictions and model disagreements, positioning TruthLens as a decision-support tool rather than an authoritative truth oracle.

Table of Contents

Introduction.....	1
Project Briefing.....	1
AI Implementation	2
Aims.....	4
Objectives	5
Artefact	5
Academic Question.....	6
Scope and Limitations of the Project	7
Report Structure	8
Literature Review.....	9
Research Papers Review	9
Abedalla et al. (2021) — 'A Closer Look at Fake News Detection: A Deep Learning Perspective'	9
Mridha et al. (2021) — 'A Comprehensive Review on Fake News Detection With Deep Learning'	9
Pavlov and Mirceva (2022) — 'COVID-19 Fake News Detection by Using BERT and RoBERTa Models'	9
Suryavardan et al. (2023) — 'Factify 2: A Multimodal Fake News and Satire News Dataset'	10
Jain (2018) — 'Fake News Detection' (IEEE)	10
Singhal et al. (2022) — 'Inter-modality Discordance for Multimodal Fake News Detection'	10
Zhang et al. (2024) — 'Multimodal Fake News Analysis Based on Image-Text Similarity'	10
Giachanou et al. (2022) — 'Multimodal Multi-image Fake News Detection'	11
Safdar and Wasim (2024) — 'RAFAH: RoBERTa Fine-Tuned Model and a Hybrid Features Set for Fake News Detection'	11
Gautam and Timilsina (2022) — 'NepBERTa: Nepali Language Model Trained in a Large Corpus'	11
Existing Systems Comparison	11
AI Algorithm Discussion and Comparison.....	12
Project Methodology.....	13
Technology and Tools Used for the Project	16

Artefact Designs.....	21
Deliverable 1: AI Model and Backend API.....	21
Software Requirements Specification (SRS).....	21
Design and Modelling Diagrams	22
Testing.....	29
Deliverable 2: News Verification Web Portal and Browser Extension.....	31
Software Requirements Specification (SRS).....	31
Design and Modelling Diagrams	32
Testing.....	42
AI-Specific Artefact Documentation	45
Data Collection	45
Model Development.....	48
Optimisation and Evaluation.....	49
AI Integration into the Application.....	51
Comparing Algorithm Performance (Model A vs Model B vs Hybrid).....	52
AI Testing — Accuracy Visualisations	53
Confusion Matrix	55
ROC Curve.....	56
Conclusion	58
Critical Evaluation of the Project.....	60
Self-Reflection	61
Evidence of Project Management	62
Log Sheets — Signed and Scanned by Supervisor	64
References and Bibliography	75
Appendices.....	77
Appendix A: Mathematical Derivations	77
Appendix B: User Manual	77
Appendix C: System Configuration.....	78

Table of figures

Figure 1: Functional Decomposition Diagram (FDD).....	6
Figure 2: Mid Gantt Chart.....	15
Figure 3: System Architecture Diagram	23
Figure 4: Sequence Diagram 1.....	24
Figure 5: Sequence Diagram 2.....	24
Figure 6: Activity Diagram 1	25
Figure 7: Activity Diagram 2.....	26
Figure 8: Activity Diagram 3.....	27
Figure 9: Activity Diagram 4.....	28
Figure 10: Class Diagram	29
Figure 11: Use Case Diagram	32
Figure 12: Landing page wireframe.....	33
Figure 13: Fake News Detection Wireframe	34
Figure 14: Login Page Wireframe	34
Figure 15: Signup Page Wireframe.....	35
Figure 16: Landing page Webpage.....	36
Figure 17: Fake News Detection WebPage	37
Figure 18: Settings page webpage	38
Figure 19: Login Page webpage	39
Figure 20: Signup Page webpage.....	39
Figure 21: Entity Relationship Diagram (ERD)	40
Figure 22: Sequence Diagram 1 (Text-Based Verification Flow)	41
Figure 23: Sequence Diagram 2 (URL-Based Verification Flow)	41
Figure 24: Extension State Diagram (ESD).....	42
Figure 25: Fake News Detection Testing.....	44
Figure 26: Fake news Detection extension testing	44
Figure 27: Dataset Class Balance	48
Figure 28: Training vs Validation Loss Curve	50
Figure 29: Validation Accuracy Curves per Epoch	50
Figure 30: Accuracy Metrics	53
Figure 31: macro Precision, Recall and F1-Score	54
Figure 32: Class-Wise Precision, Recall and F1-Score	54
Figure 33: Confusion Matrix (Model A)	55
Figure 34: Confusion Matrix (Model B).....	56
Figure 35: ROC Curve.....	57
Figure 36: Precision-Recall Curve.....	57
Figure 37: Full Gantt Chart.....	74

Introduction

Project Briefing

The way information spreads has changed dramatically because of digital technology. Many people worldwide read stories on social media sites, messaging apps and online news aggregators before they can be thoroughly reviewed and checked by an editor. There is evidence to suggest that misinformation on many topics, including health, politics and finance, can mislead the public. Widespread misinformation also undermines trust in genuine journalism and can affect the democratic process. Fact-checking organizations, such as PolitiFact, Snopes and Full Fact, do a significant amount of great work validating claims of people; unfortunately, their ability to validate is limited when compared to the ever-increasing amount of content produced daily.

TruthLens fills this void by providing an AI-based verification platform that provides users with rapid and probabilistic assessments of a claim's credibility. This platform has been designed to support human decision-making rather than replace it by providing users with well-structured signals about the content they consume and/or share, such as credibility data for the source, confidence score of the model, and explanations of each token in the response. Users can utilize this platform through two different interaction modes: the full web application providing them with room for in-depth analysis, and the lightweight Chrome browser extension that enables the user to validate the credibility of content while they are web browsing.

The project involves creating an application of Artificial Intelligence (AI) for Natural Language Processing (NLP). The main function of the application is to perform Binary Text Classification (i.e., REAL vs. FAKE) by applying supervised machine learning to fine-tuned transformer models. Through an iterative development process, the lessons learned from this project have helped to inform the development of the overall architecture, including the importance of using quality datasets, avoiding shortcut learning, and using hybrid detection methodologies.

AI Implementation

AI Aspect Addressed

- Natural Language Processing (NLP): Text classification on news sources is the main area of AI addressed through Natural Language Processing (NLP). The system leverages both short and long-form text (headlines and claims versus complete articles) to assign ratings for a given text's credibility.
- Machine Learning (Supervised): The training methods for both Model A and Model B are based on supervised learning techniques using labelled datasets containing human-verified REAL or FAKE designations because of fact-checking evaluations.
- Explainable AI (XAI): Explainable AI (XAI) allows the integration of the LIME (Local Interpretable Model Agnostic Explanations) methodology to produce tokenized feature importance scores, enhancing the interpretability and transparency of model predictions for the end-user.

Learning Paradigm and Justification

The decision to implement a supervised learning method is appropriate given that there is a clearly defined predetermination for the classification: either being REAL or FAKE according to an Original Authoritative Fact Check (as noted, with backing from a fact-checking agency). In addition, many publicly available, labeled datasets (LIAR, FakeNewsNet, cleaned ISOT/diverse corpora, curated synthetic/targeted sets) provide excellent training data that can be used to fine-tune pre-trained language models, making the training signals for these methods reliable. While unsupervised/self-supervised methodologies were evaluated, those methodologies did not utilize the factual labels that are critical to how the project will be evaluated on its distinctiveness from other projects. The use of a supervised methodology will also make it easier to apply Standard Classification Metrics (Accuracy, Precision, Recall, F1 Score, ROC-AUC) for quantitatively evaluating the models built by this project.

Mathematics Behind AI and Model Flow

The mathematical foundation of the system rests on several key concepts from deep learning and NLP:

Tokenisation and Embedding: Input text is first tokenised into subword units using the Byte-Pair Encoding (BPE) tokenizer associated with RoBERTa. Each token is mapped to a dense embedding vector in a d-dimensional space (d=768 for RoBERTa-base). Positional encodings are added to preserve sequence order information.

Transformer Encoder: The embedded tokens are processed through 12 layers of multi-head self-attention and feed-forward networks. Self-attention computes: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$, where Q, K, V are linear projections of the input, enabling the model to capture contextual relationships between all token pairs.

Classification Head: The [CLS] token representation from the final transformer layer is passed through a linear classification head that produces logits $z = [z_0, z_1]$ for the two classes (REAL, FAKE).

Softmax and Probability: The logits are converted to probabilities using the softmax function: $P(\text{class}_i) = \exp(z_i) / \sum(\exp(z_j))$, yielding calibrated confidence scores for each class.

Cross-Entropy Loss: During training, model parameters are optimised by minimising the binary cross-entropy loss: $L = -[y * \log(p) + (1-y) * \log(1-p)]$, where y is the ground-truth label and p is the predicted probability.

LoRA (Low-Rank Adaptation): Rather than fine-tuning all 125 million parameters of RoBERTa-base, LoRA injects trainable low-rank decomposition matrices into selected attention layers. For a pre-trained weight matrix W, the update is parameterised as: $W' = W + BA$, where B is an (d x r) matrix and A is an (r x d) matrix, with rank $r \ll d$ (r=16 in this project). This reduces trainable parameters to approximately 1% of the full model while maintaining competitive performance.

A) Self-Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

B) SoftMax Function

$$P(\text{class}_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

C) Binary Cross-Entropy Loss

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

D) LoRA Decomposition

$$W' = W + BA$$

Agent Description

As per the PEAS model for software agents, the TruthLens System can be defined as a Decision Support Agent that functions within the domain of Information Verification. Its performance (P) is based on its ability to correctly classify news articles, as measured by its classification accuracy, F1-Score, and the quality of its uncertainty signals. Its environment (E) is the Entire Internet that has a multitude of different news articles that can vary significantly in terms of their length, style, and level of credibility. Its actuators (A) produce structured API responses which include verdicts, risk levels, step traces, and LIME explanations. Its sensors (S) are the source domain metadata and the text associated with the input (which could either be in the form of an actual article or extracted from a URL).

Within the agent's operating methodology, it executes the following steps sequentially: It will first check the credibility of the source around which the article is published; then it will use Model A, to assess the validity of the headline/claim's text; then it will use Model B to assess the article's body text; it will then use the conflict resolution and confidence thresholds to determine an overall composite verdict for that particular article. In addition, the agent was designed to communicate uncertainty rather than create a forced binary decision when it lacks enough confidence to issue a confident decision. This design criterion reflects an honest acknowledgment of the limitations of its models and the information contained in the articles.

Aims

The project's overall goal is to develop, implement, and assess an AI-enabled fake news verification system that will give timely, understandable, and credible evaluations of English language news. Users should be able to use structured analysis signals to support responsible use of information. Therefore, when they receive online content, they will have the ability to make well-informed choices about it.

Objectives

1. To develop a web-based verification platform that integrates transformer-based fake news detection models capable of analyzing both short-form text (headlines/claims) and long-form articles.
2. To support multiple input methods—including manual text entry and URL-based input with automatic article content extraction—providing flexibility for different verification scenarios.
3. To provide real-time classification outputs including a verdict (Suspicious, Uncertain, or Likely Real), a risk level, a step-by-step analysis trace, and explicit uncertainty handling for low-confidence or conflicting predictions.
4. To implement explainable AI (XAI) outputs using LIME to highlight the most influential text features driving each prediction, improving transparency and user trust.
5. To develop a browser extension that enables rapid in-context verification during normal web browsing, extending the platform's accessibility beyond the main web application.
6. To evaluate the system using established classification metrics (accuracy, precision, recall, F1-score, confusion matrices, ROC curves) and structured error analysis, documenting both capabilities and limitations honestly.

Artefact

The primary artefact is the TruthLens platform, a modular fake news detection system composed of interconnected subsystems. The platform is divided into two major deliverables as defined in the Project Registration Form (PRF):

Artefact 1: Fake News Detection Model and Backend API — A fine-tuned RoBERTa model (delivered as two specialised adapters: Model A for headlines and Model B for articles) deployed in a Python FastAPI backend with endpoints for text-based classification and URL-based article extraction and analysis. The backend implements a three-layer hybrid detection pipeline (source credibility check, headline analysis, and article analysis) with conflict resolution, uncertainty handling, and LIME-powered explainability.

Artefact 2: News Verification Web Portal and Browser Extension — A Next.js 16 web application with user authentication (NextAuth + MongoDB), a news aggregation homepage with credibility signals, a dedicated fake news detection page with interactive analysis results, and a Chrome browser extension (Manifest V3) for popup-based verification. The portal

provides user preferences management, responsive design, and progressive disclosure of analysis details.

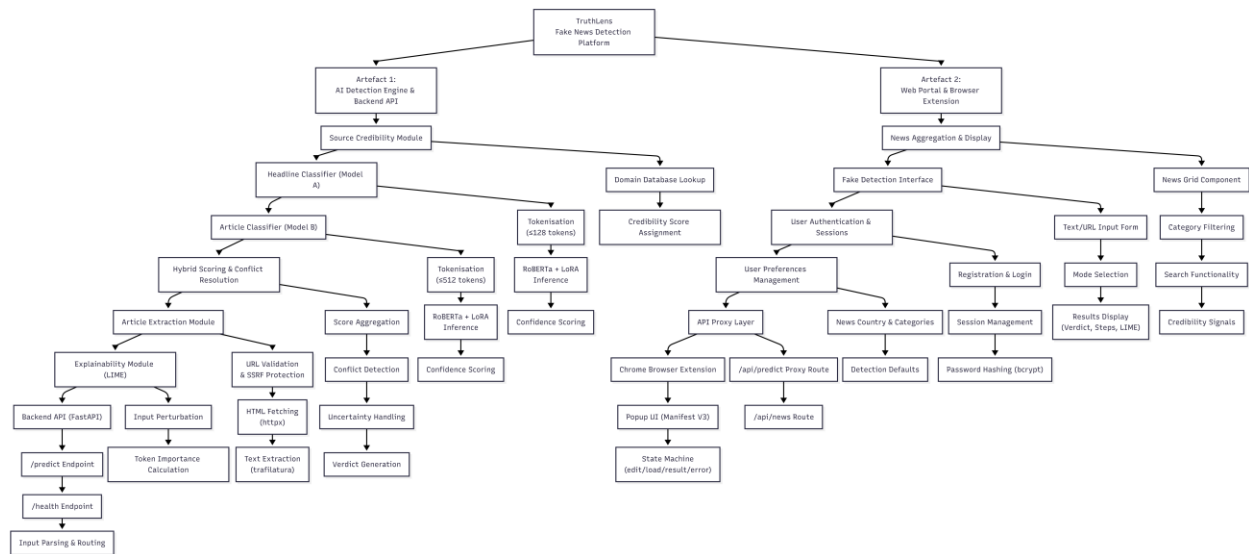


Figure 1: Functional Decomposition Diagram (FDD)

Academic Question

Academic Question: What is an ideal system architecture for a real-time, web-based fake news detection system that uses a fine-tuned RoBERTa model for classification, outputs explainable AI (XAI), uses multiple input methods (text and URL), and provides precise classifications that instil trust among users?

This question is addressed throughout the report by documenting: (1) the architectural decisions behind the modular three-layer hybrid pipeline and why this design improves robustness over single-model approaches; (2) the choice of RoBERTa-base with LoRA as a balance between performance and resource efficiency; (3) the integration of LIME for post-hoc explainability and how token-level feature importance is communicated to users; (4) the explicit uncertainty handling mechanisms (conflict detection, low-confidence flagging) that prevent overconfident predictions and support user trust; and (5) the evaluation evidence demonstrating system behaviour under controlled batch tests and real-world URL extraction scenarios.

A supervised learning approach is justified because the task has a well-defined target output—a discrete binary label (REAL or FAKE)—derived from authoritative fact-checking annotations. The availability of multiple labelled datasets (LIAR, FakeNewsNet, cleaned ISOT/diverse corpora, and curated synthetic/targeted sets) provides sufficient training signal for fine-tuning pre-trained language models. Unsupervised or self-supervised approaches were considered but deemed less suitable for this application because they would not leverage the fact-checking labels that are central to the project's evaluation methodology. The supervised framing also aligns naturally with established classification metrics (accuracy, precision, recall, F1-score, ROC-AUC) that enable rigorous quantitative evaluation.

Scope and Limitations of the Project

Scope (Inclusions):

- English-language fake news detection for text inputs (headlines, claims, full articles) and URL-extracted article content.
- A web-based user interface built with Next.js 16, providing news aggregation with credibility signals, a dedicated detection tool, and user account management.
- A FastAPI backend serving a hybrid detection pipeline with two transformer-based models, source credibility checks, and LIME explainability.
- A Chrome browser extension (Manifest V3) enabling popup-based verification during browsing.
- Quantitative evaluation using accuracy, precision, recall, F1-score, confusion matrices, and ROC curves on curated batch test sets.
- Documentation of the complete model training journey, including failed approaches, challenges encountered, and solutions implemented.

Limitations and Exclusions:

- Detection of fake news in languages other than English (Nepali language support is identified as future work).
- Detection of multimedia-based misinformation such as deepfakes, manipulated images, or altered videos.

- Real-time fact-checking against live knowledge bases or fact-checking APIs (e.g., Google Fact Check Tools API).
- Large-scale production deployment with load balancing, horizontal scaling, or CDN distribution—the system is delivered as a functional prototype.
- Advanced security features beyond basic authentication (e.g., two-factor authentication, OAuth social login).
- URL extraction is limited by website accessibility; paywalled, JavaScript-heavy, or access-restricted pages may not yield extractable content.

Report Structure

The remainder of this report proceeds as follows. It begins with literature review surveying research on transformer-based fake news detection, dataset-related challenges, and explainability techniques. This is followed by a justification of the selected project methodology and a detailed presentation of the technology stack, including rationale for each component choice.

Subsequent chapters document the artefact designs, covering system architecture, data collection processes, model development, testing evidence, and AI-specific evaluation metrics such as confusion matrices, ROC curves, and A/B model comparisons. The report then presents the overall conclusions, explicitly linking findings back to the stated aims, objectives, and academic research questions. A critical evaluation and personal reflection are provided thereafter.

The final part of the document includes evidence of project management activities, including log sheets and the Gantt chart, followed by the full reference list and appendices containing supplementary material.

Literature Review

This literature review surveys the current state of research in automated fake news detection, with particular emphasis on text-based approaches using deep learning, the challenges of dataset quality and model robustness, and the role of explainability in fostering user trust. The review draws from ten research papers and examines five existing detection systems to establish the theoretical and practical foundation upon which TruthLens is built.

Research Papers Review

Abedalla et al. (2021) — 'A Closer Look at Fake News Detection: A Deep Learning Perspective'

This study provides a comparative analysis of deep learning architectures for fake news detection, including CNNs, LSTMs, and transformer-based models. The authors demonstrate that transformer models (particularly BERT variants) consistently outperform traditional architectures on benchmark datasets. However, they highlight the risk of overfitting when models are trained on datasets with strong stylistic biases. This finding directly influenced the decision in TruthLens to move away from the ISOT dataset after identifying shortcut learning issues, and to adopt multi-source training data for improved generalisation.

Mridha et al. (2021) — 'A Comprehensive Review on Fake News Detection With Deep Learning'

This comprehensive survey categorises fake news detection approaches into content-based, social context-based, and hybrid methods. The authors argue that content-based approaches using transformer encoders represent the current state of the art for text classification but emphasize that real-world deployment requires attention to domain shift and dataset artefacts. Their recommendation for hybrid approaches combining multiple detection signals strongly influenced the three-layer architecture of TruthLens, which combines source credibility, headline analysis, and article analysis rather than relying on a single model.

Pavlov and Mirceva (2022) — 'COVID-19 Fake News Detection by Using BERT and RoBERTa Models'

This paper compares BERT and RoBERTa performance on COVID-19 misinformation datasets, finding that RoBERTa achieves superior accuracy due to its improved pre-training methodology (dynamic masking, larger batches, more training data). The study informed TruthLens's

emphasis on dataset diversity and domain-shift awareness. In the final checked training configuration, COVID ingestion is disabled, so COVID findings are treated as methodological guidance rather than a direct final training component.

Suryavardan et al. (2023) — 'Factify 2: A Multimodal Fake News and Satire News Dataset'

This work introduces a multimodal dataset that includes satire as a distinct category of misleading content. The authors demonstrate that satire detection remains challenging for text-only models because satire often mimics the structure of legitimate news while containing exaggerated or absurd claims. This insight influenced the addition of known satire domains to the TruthLens source credibility database, providing a rule-based first-pass filter for satirical content that text models might misclassify.

Jain (2018) — 'Fake News Detection' (IEEE)

This early work establishes baseline approaches for fake news detection using traditional machine learning (Naive Bayes, SVM with TF-IDF features). While these methods achieve reasonable accuracy on simpler datasets, the author acknowledges their limited ability to capture semantic nuances in text. This paper provides historical context for the evolution from feature-engineered approaches to end-to-end deep learning methods adopted in TruthLens and demonstrates the performance gains achieved by transitioning to transformer-based architectures.

Singhal et al. (2022) — 'Inter-modality Discordance for Multimodal Fake News Detection'

This research explores the relationship between text and image modalities in fake news, finding that discordance between modalities (e.g., misleading images paired with accurate text) is a strong indicator of misinformation. While TruthLens focuses on text-only detection, this work highlights the importance of multi-signal approaches for robust detection principle applied in TruthLens through the combination of source, headline, and article analysis layers rather than relying on a single signal.

Zhang et al. (2024) — 'Multimodal Fake News Analysis Based on Image-Text Similarity'

Zhang et al. propose using image-text similarity metrics to identify fake news where visual and textual content are inconsistent. Their findings reinforce the limitation of text-only approaches and suggest directions for future TruthLens enhancements. For the current project scope, the text-only constraint is documented as an explicit limitation, with multimodal detection identified as a potential enhancement.

Giachanou et al. (2022) — 'Multimodal Multi-image Fake News Detection'

This paper extends multimodal approaches to handle multiple images per article. Its relevance to TruthLens is primarily as a contrast—demonstrating the complexity of multimodal detection and validating the project's decision to focus on text-based detection as a well-scoped deliverable within the available timeframe while acknowledging multimodal extensions as future work.

Safdar and Wasim (2024) — 'RAFAH: RoBERTa Fine-Tuned Model and a Hybrid Features Set for Fake News Detection'

This paper demonstrates that RoBERTa fine-tuned with hybrid feature sets (combining contextual embeddings with handcrafted linguistic features) achieves strong performance on fake news benchmarks. The authors' use of RoBERTa as a backbone validates TruthLens's model selection. However, TruthLens opts for a simpler fine-tuning approach (LoRA adapters without additional handcrafted features) to maintain deployment simplicity, with the hybrid detection layers (source check, headline model, article model) providing the complementary signals that handcrafted features might otherwise supply.

Gautam and Timilsina (2022) — 'NepBERTa: Nepali Language Model Trained in a Large Corpus'

This paper presents NepBERTa, a BERT-based model pre-trained on Nepali text corpora. While not directly related to English fake news detection, this work is relevant to TruthLens's stated future work goal of extending detection to low-resource languages like Nepali. The existence of NepBERTa demonstrates that the LoRA fine-tuning approach used in TruthLens could potentially be applied to NepBERTa for Nepali fake news detection, providing a concrete pathway for multilingual expansion.

Existing Systems Comparison

- **Google Fact Check Tools:** Provides a searchable database of fact-checked claims by verified organizations. TruthLens differs by offering real-time AI classification rather than lookup-based verification, enabling analysis of novel claims not yet fact-checked by human reviewers.
- **ClaimBuster:** An automated claim detection system that identifies check-worthy claims in text. While ClaimBuster focuses on identifying claims that merit factchecking, TruthLens focuses on providing a credibility assessment for claims already identified by the user.

- **Fake News Challenge (FNC-1):** A stance detection shared task that classifies headline-body pairs as agrees/disagrees/discusses/unrelated. TruthLens's dual-model approach (Model A for headlines, Model B for bodies) is conceptually related but makes independent predictions rather than stance classification.
- **Full Fact Alpha:** An automated fact-checking prototype by the UK's Full Fact organization. Full Fact emphasises claim matching against known facts, while TruthLens uses pattern-based classification—a different but complementary approach.
- **Logically.ai:** A commercial fact-checking platform combining AI and human analysts. TruthLens is positioned as an academic prototype exploring similar hybrid approaches but at a smaller scale and without the human-in-the-loop editorial layer.

AI Algorithm Discussion and Comparison

The literature reveals several key themes relevant to TruthLens's design decisions. First, transformer-based models (particularly BERT and RoBERTa variants) consistently outperform traditional ML approaches and earlier neural architectures for text classification tasks, justifying the selection of RoBERTa-base as the backbone model. Second, parameter-efficient fine-tuning methods such as LoRA, Adapters, and Prefix Tuning offer a practical alternative to full fine-tuning, reducing computational requirements while maintaining competitive performance—critical for a student project with limited GPU resources. Third, dataset quality and leakage prevention are as important as model architecture; several studies report inflated accuracy metrics due to dataset artefacts rather than genuine detection capability. This theme directly informed the extensive data cleaning and diagnostic work documented in Section 9. Finally, explainability methods such as LIME and SHAP are increasingly recognised as essential components of trustworthy AI systems, particularly for applications where users must make decisions based on model outputs—motivating the integration of LIME into TruthLens's analysis pipeline.

Comparing algorithm performance from the literature: traditional ML baselines (TF-IDF + SVM/Logistic Regression) typically achieve 85-92% accuracy on standard benchmarks but struggle with nuanced semantic content. CNN/LSTM architectures improve to 90-95% but require careful feature engineering. Transformer models (BERT, RoBERTa, DeBERTa) achieve 95-99% on clean benchmarks, though these numbers can be misleading when dataset artefacts

enable shortcut learning—a problem TruthLens encountered and resolved during development. The mathematical advantage of transformers lies in the self-attention mechanism's ability to model long-range dependencies: unlike RNNs where information must flow sequentially through hidden states, self-attention computes direct pairwise relationships between all tokens in $O(n^2)$ time, capturing global context that is critical for assessing the coherence and plausibility of news claims.

Project Methodology

This project adopts an Agile-inspired iterative development methodology, structured around short development cycles with continuous evaluation and refinement. The choice of Agile over traditional Waterfall or V-Model approaches is justified by the inherent uncertainty in AI/ML projects, where model performance, dataset quality, and integration complexity cannot be fully predicted upfront and require iterative experimentation.

Specifically, the project draws from Scrum principles—time-boxed iterations, incremental delivery, and regular retrospection—adapted for a solo developer context. The key reasons for selecting this approach are:

- **Uncertainty in model performance:** The training journey demonstrated that initial model choices (e.g., the ISOT dataset) could produce misleading results. An iterative approach allowed rapid pivoting when quality issues were identified, without requiring a complete project restart.
- **Integration complexity:** The system involves multiple interconnected components (AI models, backend API, frontend, extension, database). Incremental integration—delivering a working end-to-end slice early and progressively enhancing it—reduced the risk of late-stage integration failures.
- **Feedback-driven refinement:** Regular supervisor meetings and self-evaluation (documented in logsheets) provided continuous feedback that guided prioritisation of development tasks. For example, the decision to implement LIME explainability was refined based on feedback about user trust requirements.

- Risk-driven prioritisation: High-risk items such as data leakage detection, model overfitting, and URL extraction robustness were identified early and addressed in dedicated iterations, preventing them from becoming critical blockers late in the project.

The project was structured across a 47-week timeline (June 2025 – May 2026) divided into the following major phases: Project Initiation and Proposal (Weeks 1–10), Research and Literature Review (Weeks 11–20), Dataset Exploration and Initial Model Training (Weeks 15–22), Model Iteration and Hybrid System Design (Weeks 20–26), Backend and API Development (Weeks 23–30), Frontend and Extension Development (Weeks 28–34), Testing, Evaluation, and Draft Report (Weeks 33–36), and Final Report, Defense, and Submission (Weeks 37–47). Each phase involved iterative sub-cycles of implementation, testing, and refinement.

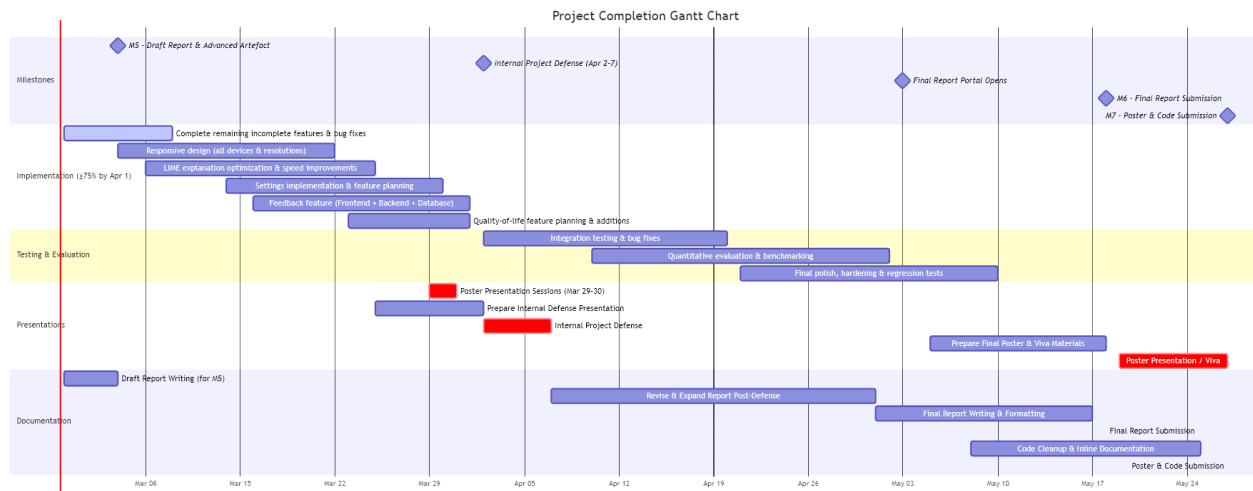


Figure 2: Mid Gantt Chart

Technology and Tools Used for the Project

The technology stack was selected to balance development productivity, ML inference requirements, ecosystem maturity, and maintainability. Each technology choice is justified below based on its specific contribution to the project goals.

Category	Technology/Tool	Justification
AI Base Model	RoBERTa-base (125M params)	Chosen over BERT-base for its improved pre-training methodology (dynamic masking, larger batches, byte-level BPE). Offers strong NLP performance while remaining computationally tractable for fine-tuning on consumer hardware.
Fine-tuning	LoRA via PEFT library	Parameter-efficient fine-tuning reduces trainable parameters to ~1% of the full model ($r=16$, $\alpha=32$), enabling training on limited GPU memory. Chosen over full fine-tuning and Adapters for its simplicity and strong empirical results.
ML Framework	PyTorch + Hugging Face Transformers	Industry-standard ecosystem for transformer model training and inference. Extensive documentation, active community, and native PEFT integration justified selection over

		TensorFlow/JAX alternatives.
Explainability	LIME (lime-text)	Selected for local, model-agnostic interpretability. Generates per-token importance scores by perturbing input text and observing prediction changes. Chosen over SHAP for its lower computational cost and simpler integration with text classification pipelines.
Backend API	FastAPI (Python)	High-performance async Python web framework with automatic OpenAPI schema generation and Pydantic validation. Chosen over Flask for its native async support, automatic documentation, and seamless integration with PyTorch inference code.
Article Extraction	trafilatura + httpx	trafilatura provides robust HTML-to-text extraction optimised for news articles, with built-in boilerplate removal. httpx provides modern async HTTP client with redirect following and timeout control. Chosen over BeautifulSoup for trafilatura's superior extraction quality on news sites.

Frontend Framework	Next.js 16 (App Router)	React-based framework with server-side rendering, API routes, and built-in routing. Chosen for its strong TypeScript support, component-based architecture, and ability to proxy API requests to the backend—simplifying CORS management.
Frontend Language	TypeScript + React 19	TypeScript provides compile-time type safety, reducing runtime errors in complex state management (e.g., prediction response handling). React 19 offers the latest improvements in rendering and state management.
Styling	TailwindCSS 4	Utility-first CSS framework enabling rapid UI development with consistent design tokens. Chosen over vanilla CSS for development speed and over CSS-in-JS solutions for better performance and smaller bundle sizes.
Authentication	NextAuth.js + MongoDB Adapter	Provides session-based authentication with configurable providers.

		MongoDB adapter persists user accounts, sessions, and preferences. Chosen for its tight Next.js integration and support for credential-based authentication.
Password Security	bcryptjs	Industry-standard password hashing library implementing the bcrypt algorithm. Chosen for its resistance to brute-force and rainbow table attacks through adaptive cost factor.
Database	MongoDB (via Mongoose)	Document-oriented NoSQL database providing flexible schema for user profiles, sessions, preferences, and audit logs. Chosen over PostgreSQL for schema flexibility during iterative development and natural JSON document compatibility with the JavaScript/TypeScript frontend.
Version Control	Git + GitHub	Distributed version control with traceable commit history. GitHub provides remote hosting, issue tracking, and collaboration features. Essential for

		professional software development practice.
IDE	Visual Studio Code	Lightweight, extensible editor with strong Python and TypeScript support, integrated terminal, Git integration, and extensive ecosystem of extensions for linting, formatting, and debugging.
Package Managers	npm (frontend) + pip (backend/AI)	npm manages JavaScript/TypeScript dependencies for the Next.js frontend. pip manages Python dependencies for the FastAPI backend and AI training pipeline. Both are standard package managers for their respective ecosystems.
Metrics/Visualisation	scikit-learn + Matplotlib + Seaborn	scikit-learn provides classification metrics (accuracy, F1, confusion matrix, ROC-AUC). Matplotlib and Seaborn generate publication-quality performance visualisation charts for evaluation and reporting.
Browser Extension	Chrome Manifest V3 (vanilla JS/CSS)	Manifest V3 is Chrome's current extension platform. Vanilla JS/CSS chose to

		minimize bundle size and avoid framework overhead in the constrained popup environment.
--	--	---

Artefact Designs

This section documents the artefact designs organized by deliverable/subsystem, following the incremental development approach. Each subsystem includes requirements, design diagrams, and testing evidence.

Deliverable 1: AI Model and Backend API

This deliverable covers the core AI component: the fake news detection models, the FastAPI backend, the article extraction module, and the LIME explainability integration.

Software Requirements Specification (SRS)

Functional Requirements:

- FR1: The system accepts text input (headline, claim, or article) and returns a structured prediction response containing verdict, risk level, final score, step-by-step analysis trace, and model outputs.
- FR2: The system shall accept a URL and attempt to extract article content using trafileatura; if extraction fails, the system shall return a structured error response with appropriate uncertainty indicators.
- FR3: The system shall support multiple input modes: auto (automatic routing), headline_only (Model A only), full_article (Model B only), and headline_plus_article (both models).
- FR4: The system shall detect conflicts between Model A and Model B when both produce high-confidence opposing predictions, flagging such cases as UNCERTAIN.
- FR5: The system shall provide LIME-based explainability outputs showing token-level feature importance when requested.
- FR6: The system shall check source URL domains against a curated credibility database and incorporate source credibility scores into the final verdict.

Non-Functional Requirements:

- NFR1: The system shall respond to prediction requests within a reasonable time frame for interactive use (subject to model inference and optional LIME computation).
- NFR2: The system shall handle invalid inputs, empty text, and extraction failures gracefully without crashing, returning structured error responses.
- NFR3: The system shall prevent SSRF attacks by validating URL hosts and blocking private/loopback IP addresses before fetching content.
- NFR4: The system shall be modular, allowing individual models to be updated or replaced without modifying the overall pipeline structure.

Design and Modelling Diagrams

System Architecture - Fake news Detection Platform

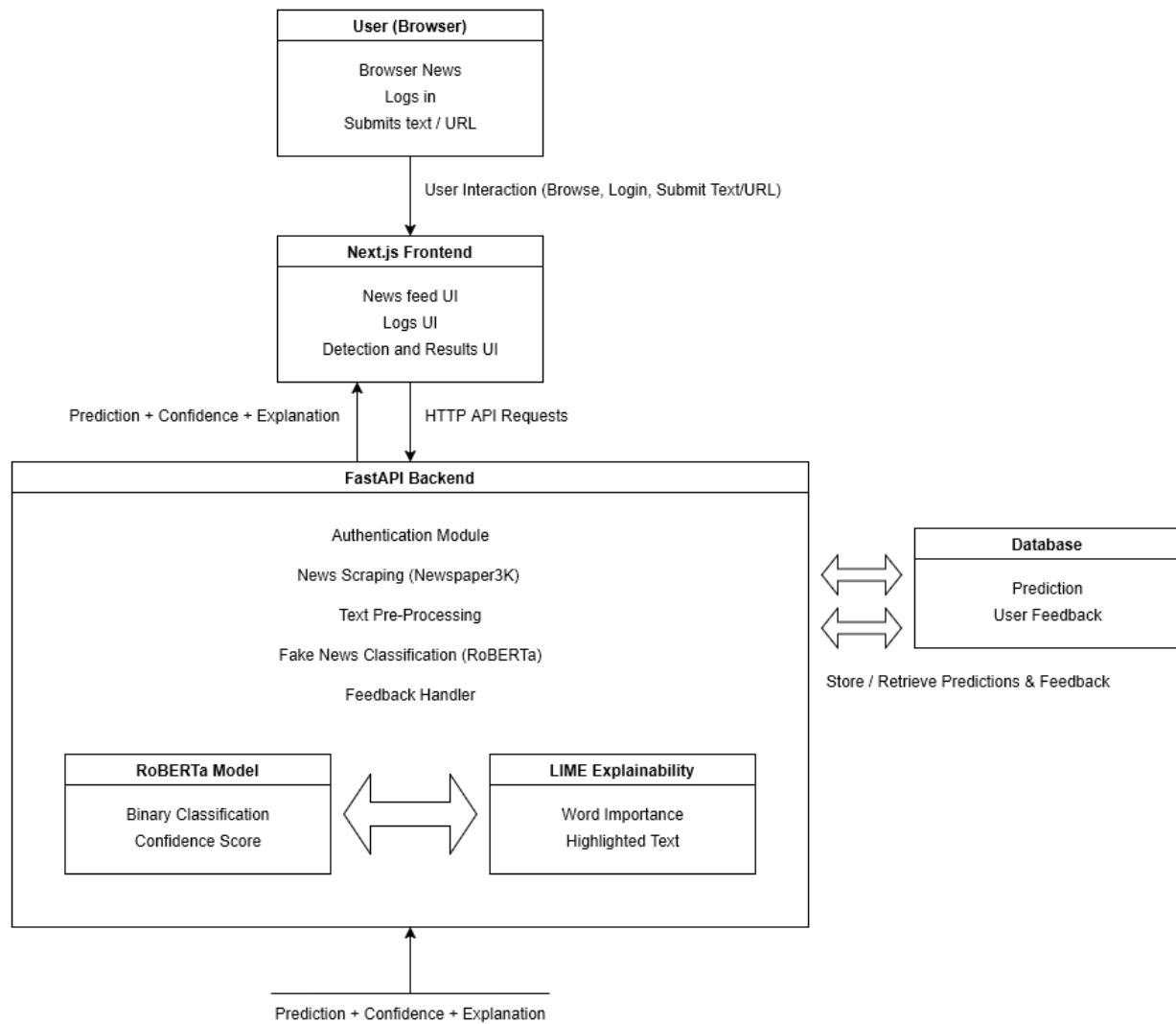


Figure 3: System Architecture Diagram

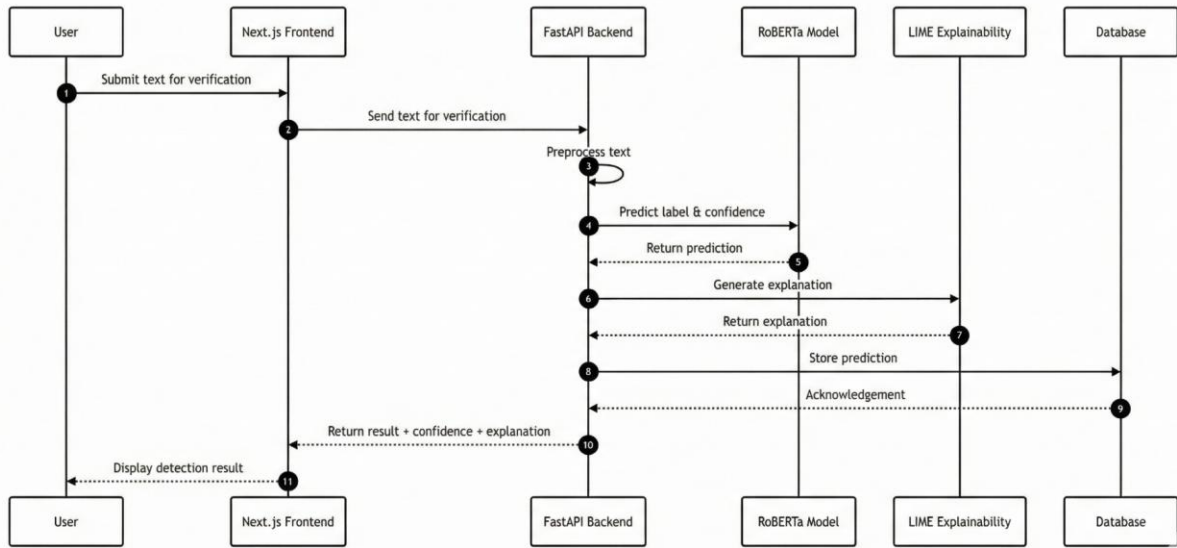


Figure 4: Sequence Diagram 1

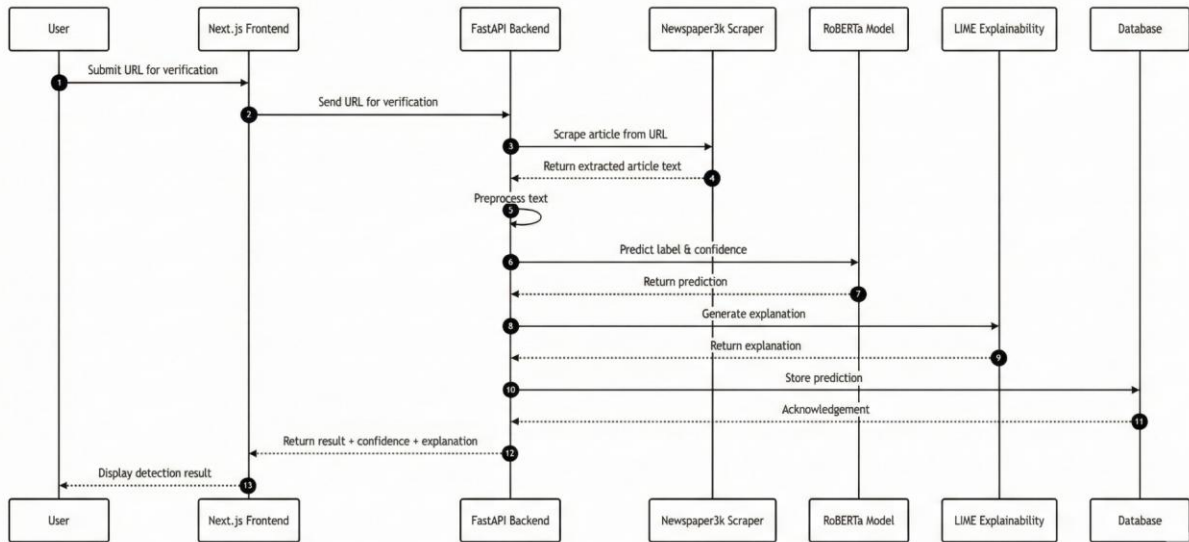


Figure 5: Sequence Diagram 2

Activity Diagram 1 - Registration & Login

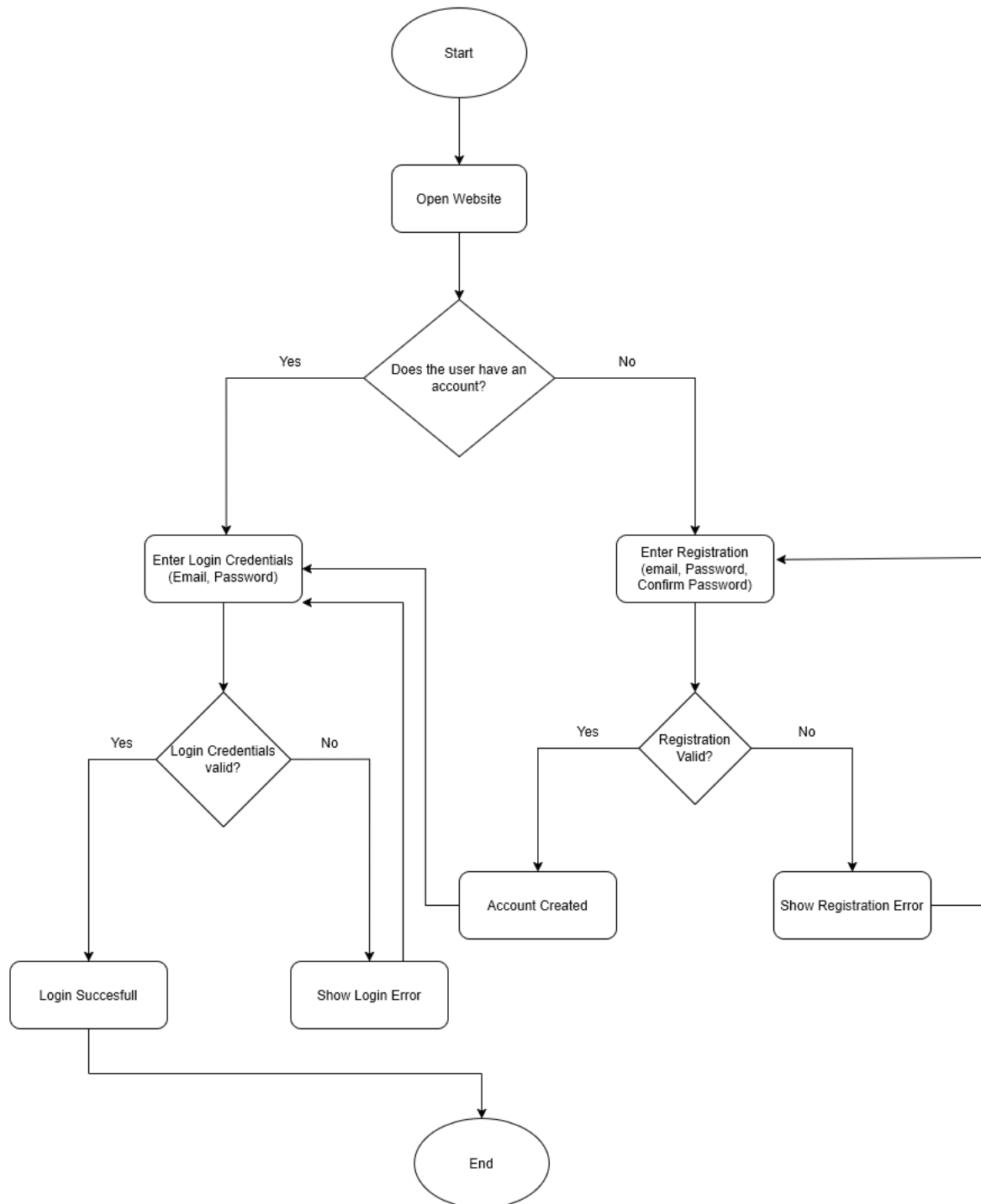


Figure 6: Activity Diagram 1

Activity Diagram 2 - Text-Based Fake News Detection

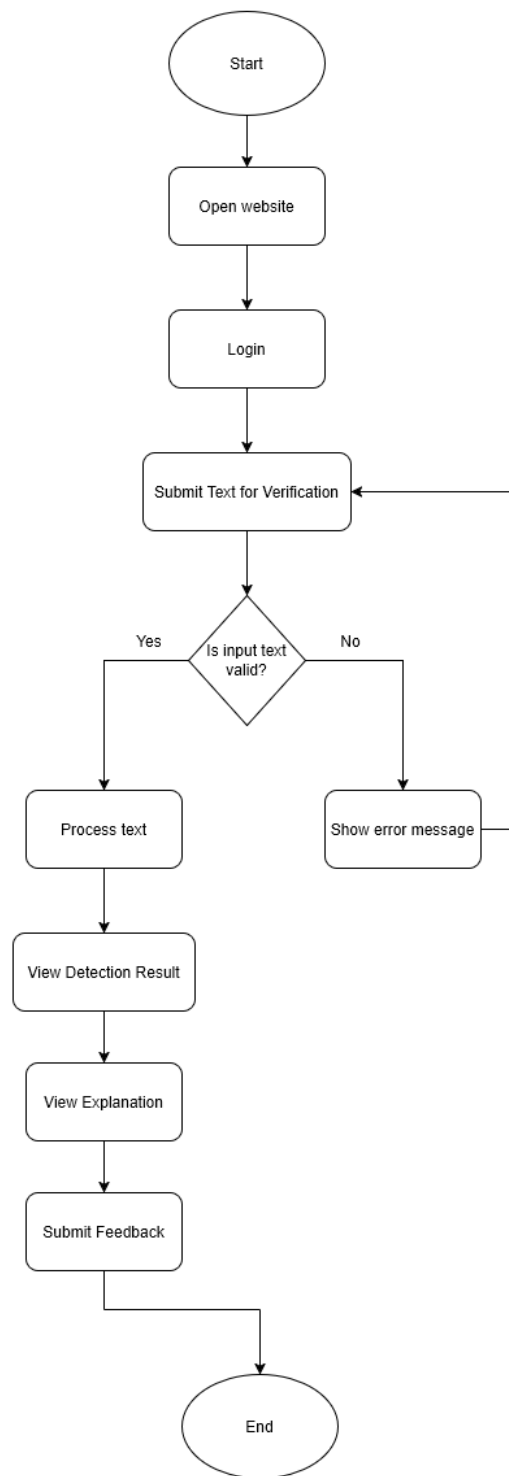


Figure 7: Activity Diagram 2

Activity Diagram 3 - URL-Based Fake News Detection

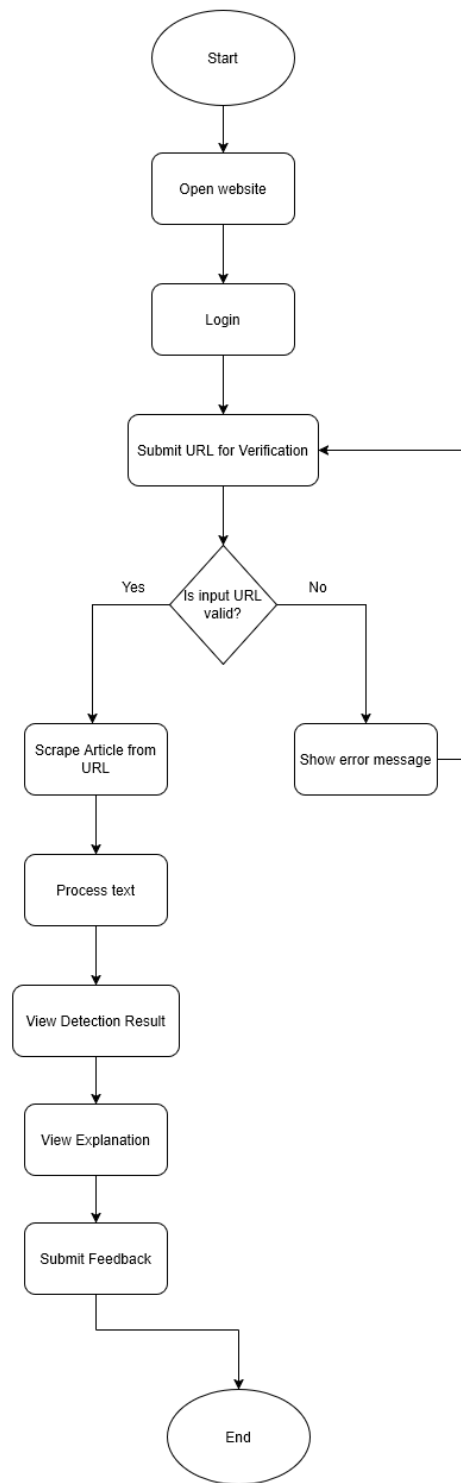


Figure 8: Activity Diagram 3

Activity Diagram 4 - Feedback Submission

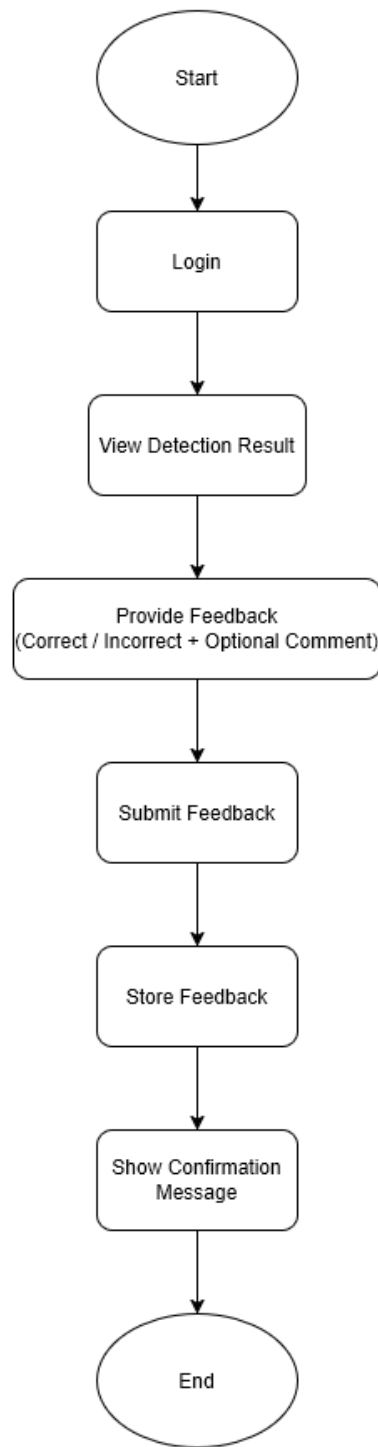


Figure 9: Activity Diagram 4

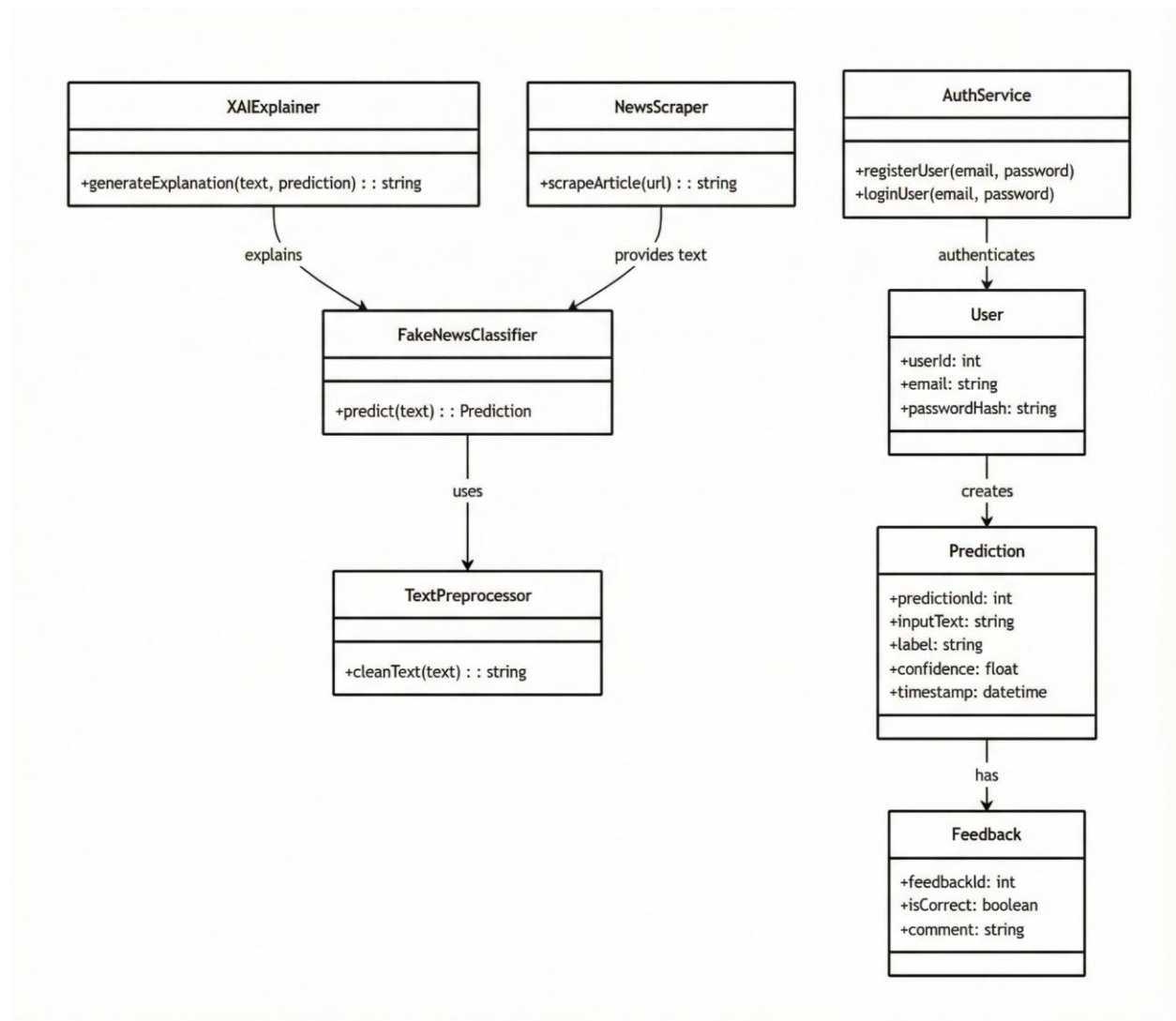


Figure 10: Class Diagram

Testing

Model-level testing was performed through curated batch tests. The following tables summarize quantitative results:

Batch Test Results Summary

Model	Input Type	Batch Size	Accuracy	Macro F1	Key Observations
Model A	Headlines/claims	101	94.06%	94.03%	No REAL misclassified as FAKE; 6

					FAKE items predicted REAL
Model B	Full articles	200	97.50%	97.04%	2 REAL misclassified as FAKE; 3 FAKE items predicted REAL

Confusion Matrices

Model A Confusion Matrix (rows = actual, columns = predicted):

	Predicted REAL	Predicted FAKE
Actual REAL	51	0
Actual FAKE	6	44

Model B Confusion Matrix (rows = actual, columns = predicted):

	Predicted REAL	Predicted FAKE
Actual REAL	58	2
Actual FAKE	3	137

Deliverable 2: News Verification Web Portal and Browser Extension

This deliverable covers the user-facing components: the Next.js web application, user authentication, news aggregation, the fake detection interface, and the Chrome browser extension.

Software Requirements Specification (SRS)

Functional Requirements:

- FR7: The web portal shall provide a homepage displaying aggregated news articles with credibility signals, supporting category filtering and search.
- FR8: The web portal shall provide a dedicated fake news detection page with text input area, URL input field, input mode selector, and analysis results display.
- FR9: The web portal shall display analysis results including verdict, risk level, step-by-step trace, model outputs, uncertainty indicators, and optional LIME explanations.
- FR10: The web portal shall support user registration, login, and session management using NextAuth with MongoDB for persistence.
- FR11: The web portal shall allow users to configure preferences including news country, categories, detection input mode, and explanation mode.
- FR12: The Chrome extension shall provide a popup-based interface for submitting text/URLs and viewing analysis results using the same API contract as the web portal.

Design and Modelling Diagrams

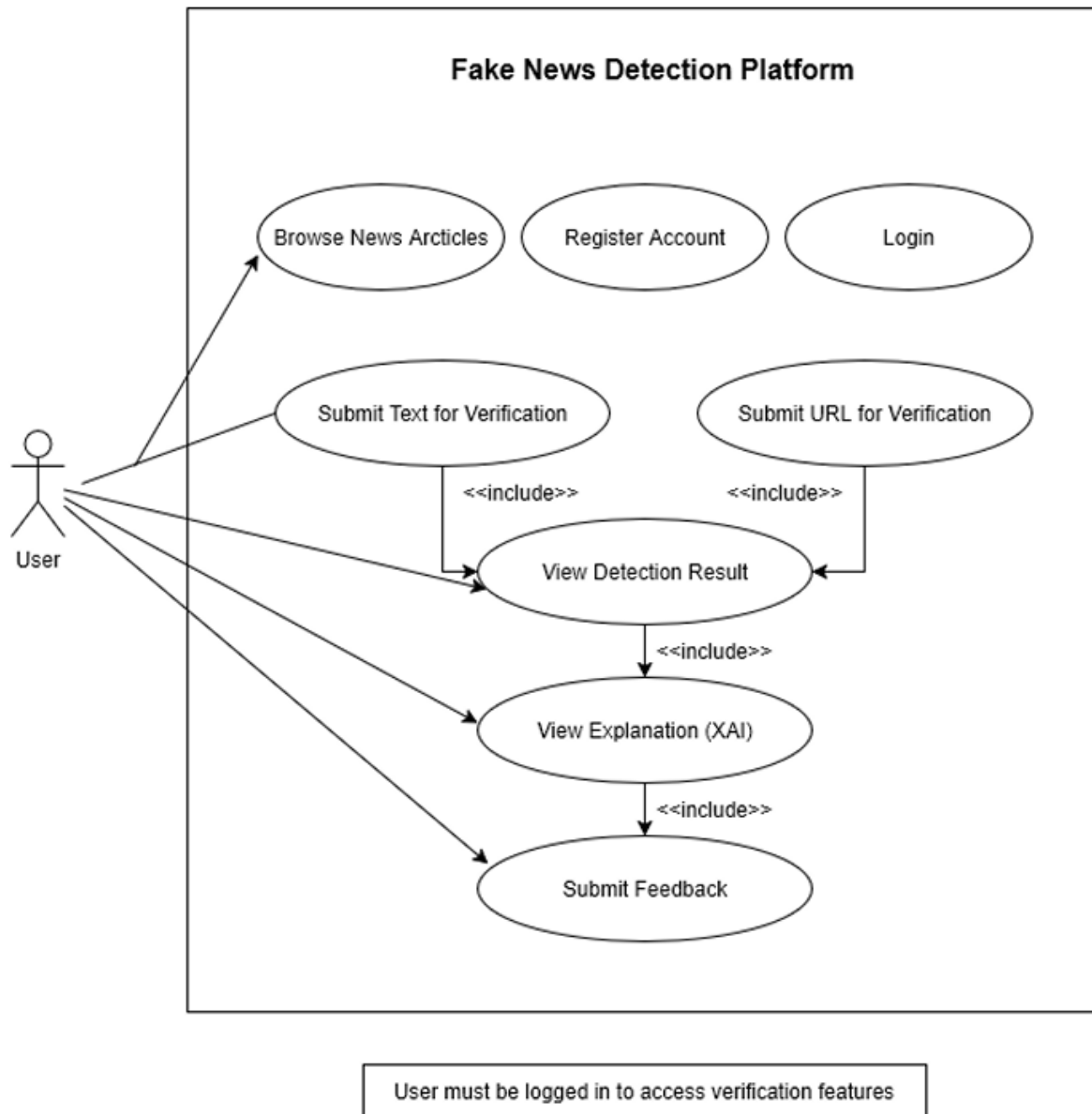


Figure 11: Use Case Diagram



Figure 12: Landing page wireframe



Figure 13: Fake News Detection Wireframe



Figure 14: Login Page Wireframe



Figure 15: Signup Page Wireframe

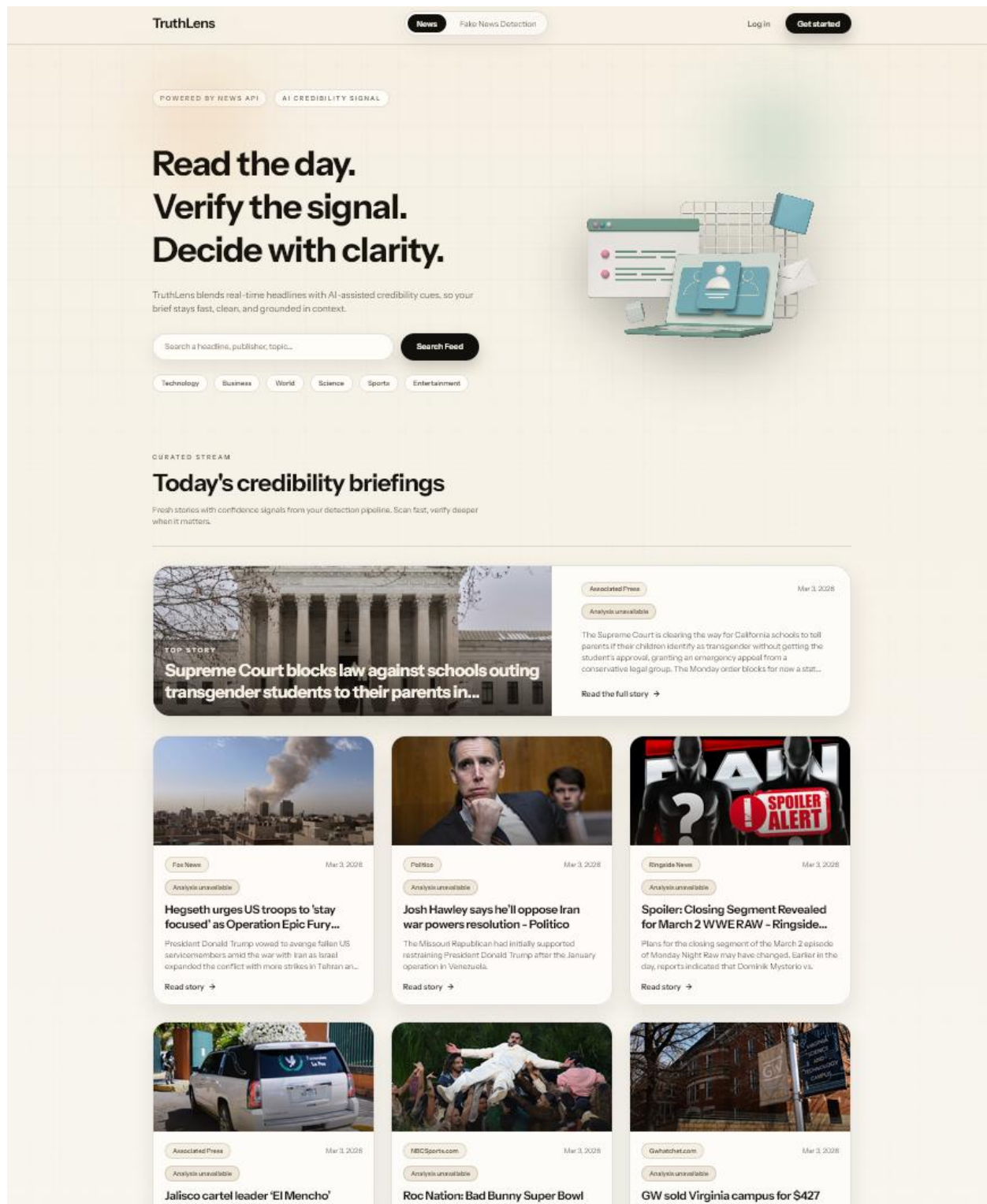


Figure 16: Landing page Webpage

TruthLens

NewsFake News Detection

R

Fake News Detection

Combine source credibility, headline signals, and deep-learning analysis to assess risk in minutes.

- Hybrid model + source credibility checks

INPUT

Article Text

Paste an excerpt or headline you want to analyse.

Paste article text here...

Input Mode

Auto detect

Auto mode is recommended. Use manual mode if your paste format is unusual.

Source URL

https://example.com

Optional: include a URL to boost source credibility scoring.

Clear fields

Analyze

Input Text or URL to detect if the news is Fake or Real

Risk: Needs Review

Hybrid credibility analysis

Paste some text and a source URL, then run an analysis to see a preview of credibility insights.

Results powered by Hybrid Deep Learning Analysis.

TruthLens

AboutContactPrivacyTerms

© 2025 TruthLens Media

Figure 17: Fake News Detection WebPage

TruthLens

News

Fake News Detection

R

Settings

Profile, security, privacy and product defaults.

Rigen Manandhar

rigenmanandhar@gmail.com

GOOGLE

Google connected, set a backup password

Required for sensitive actions

Profile

Full name

Rigen Manandhar

Email

rigenmanandhar@gmail.com

Save profile

Re-authentication

Google re-auth confirmation is required for sensitive account actions.

Verify with Google

Set backup password

New backup password

Confirm backup password

Set backup password

Preferences

Default country (2-letter)

US

business entertainment general health science sports technology

Auto

Auto explanation

Save preferences

Active sessions

Firefox on Windows (Current)

Created: 3/1/2028, 3:17:24 PM | Last seen: 3/4/2028, 1:51 PM

Revoke

Firefox on Windows

Created: 2/28/2028, 5:10:50 PM | Last seen: 3/1/2028, 2:44:17 PM

Revoke

Firefox on Windows

Created: 2/28/2028, 3:03:44 PM | Last seen: 2/28/2028, 3:19:47 PM

Revoke

Revoke all other sessions

Privacy & data

Export my data

Request export

Deletion requests are soft-deleted first, then permanently removed after 30 days.

Danger zone

Request account deletion

Privacy Terms Contact Report issue

Sign out

TruthLens

About

Contact

Privacy

Terms

© 2028 TruthLens Media

Figure 18: Settings page webpage

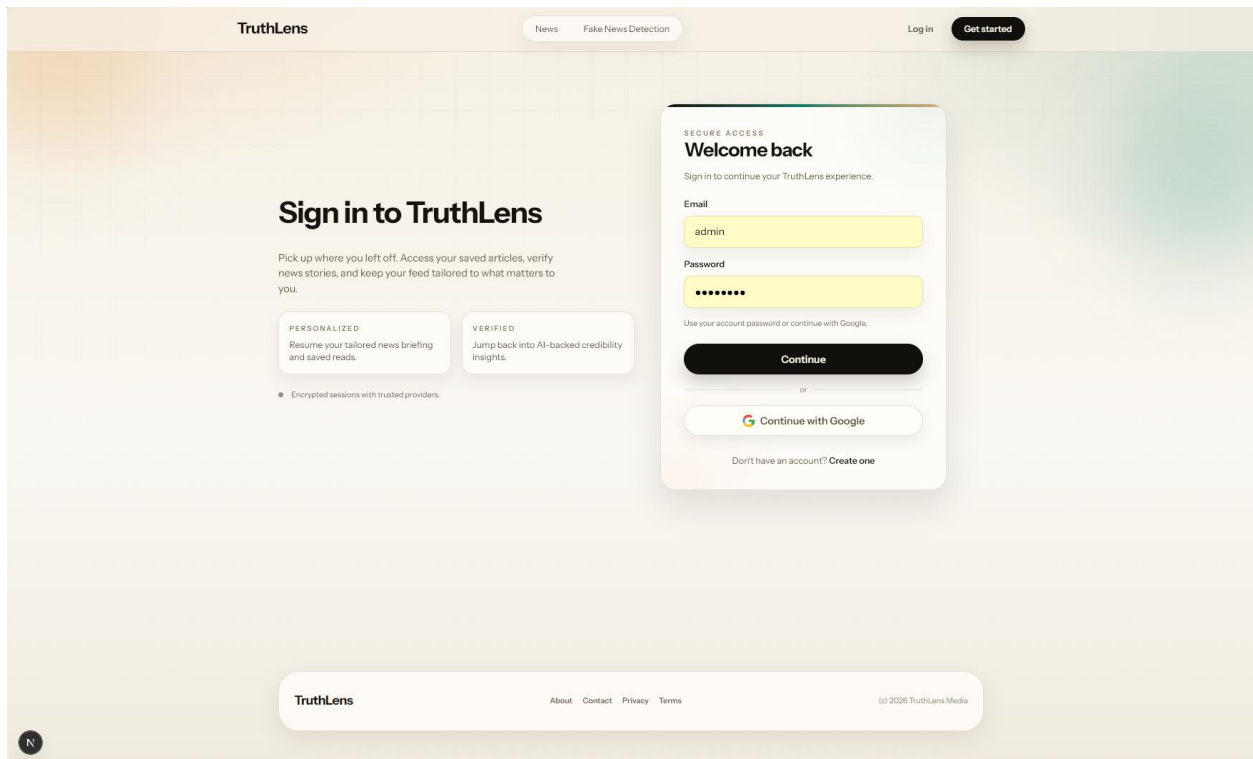


Figure 19: Login Page webpage

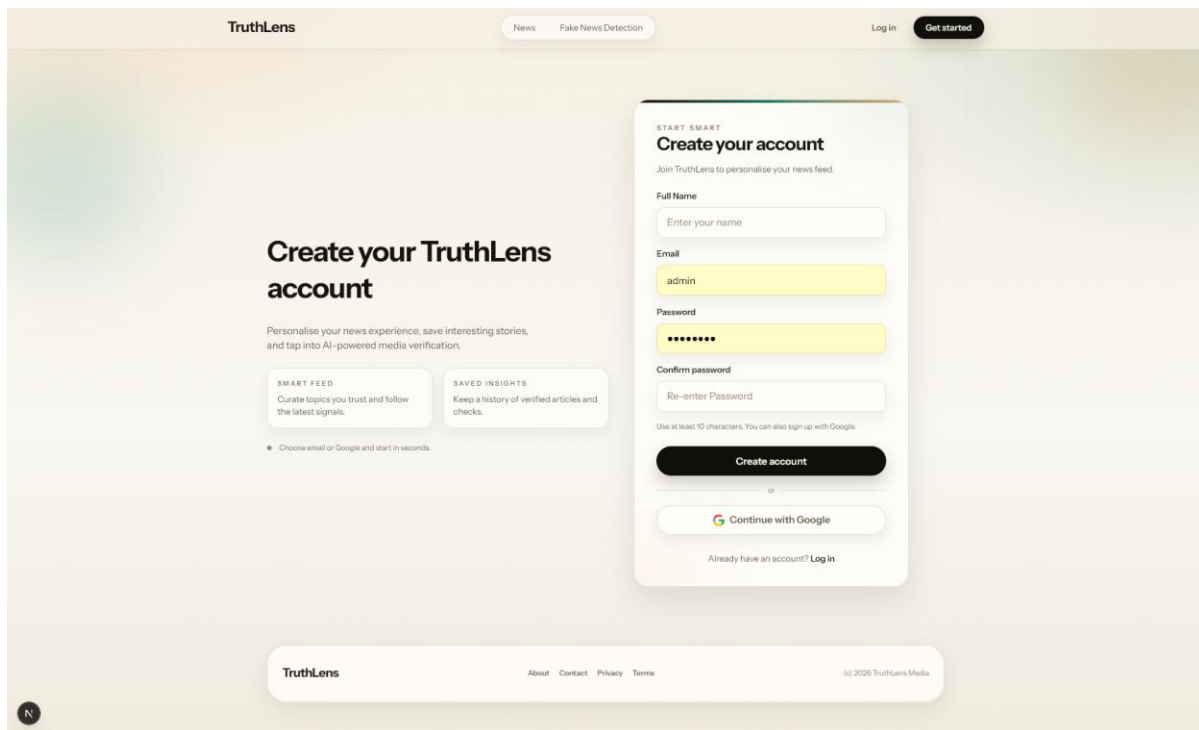


Figure 20: Signup Page webpage

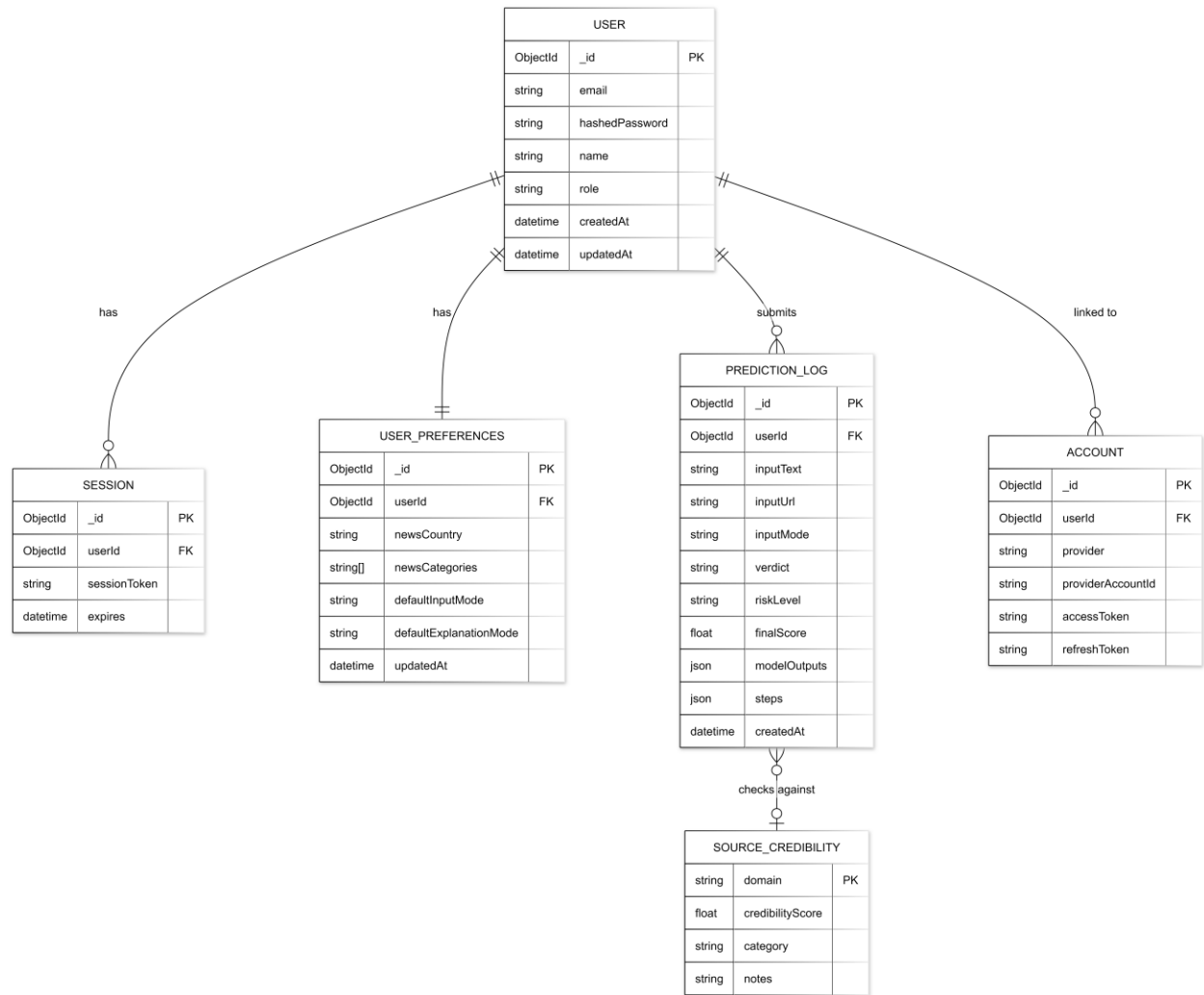


Figure 21: Entity Relationship Diagram (ERD)

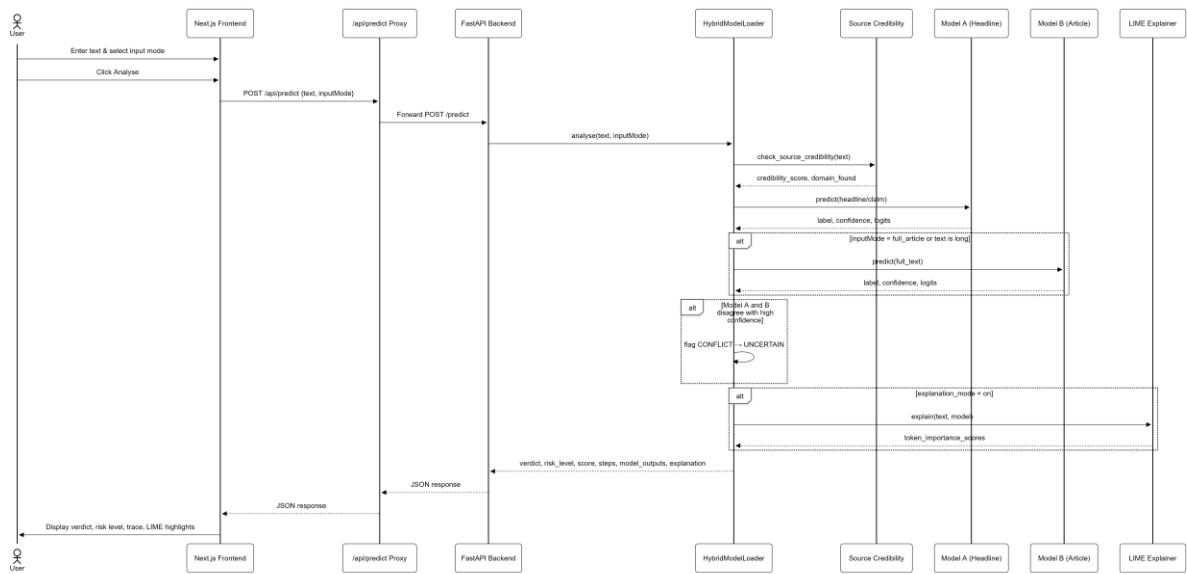


Figure 22: Sequence Diagram 1 (Text-Based Verification Flow)

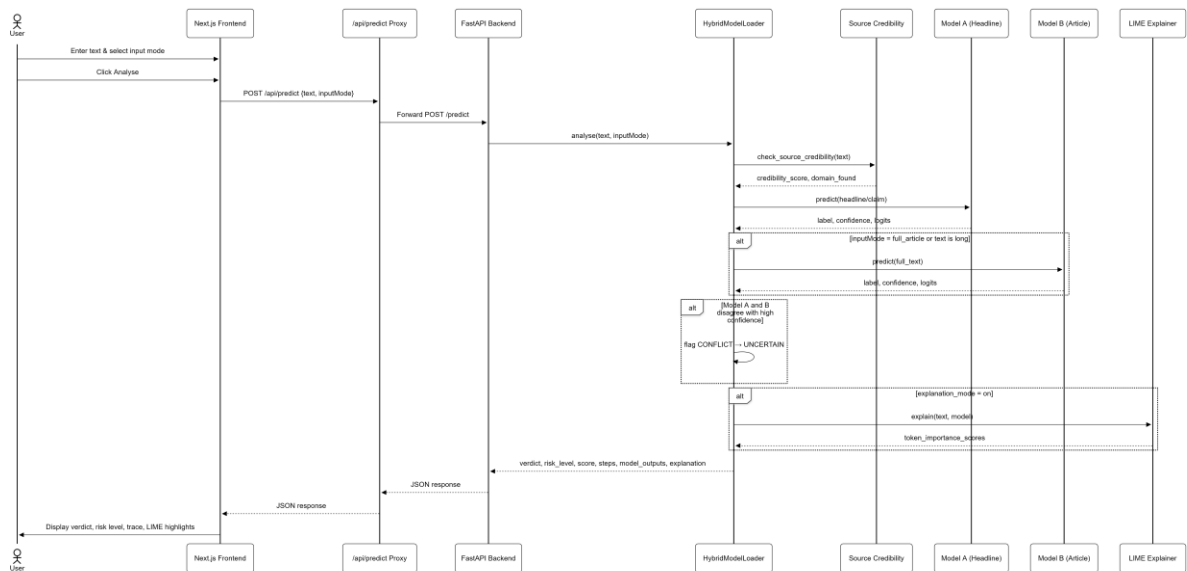


Figure 23: Sequence Diagram 2 (URL-Based Verification Flow)

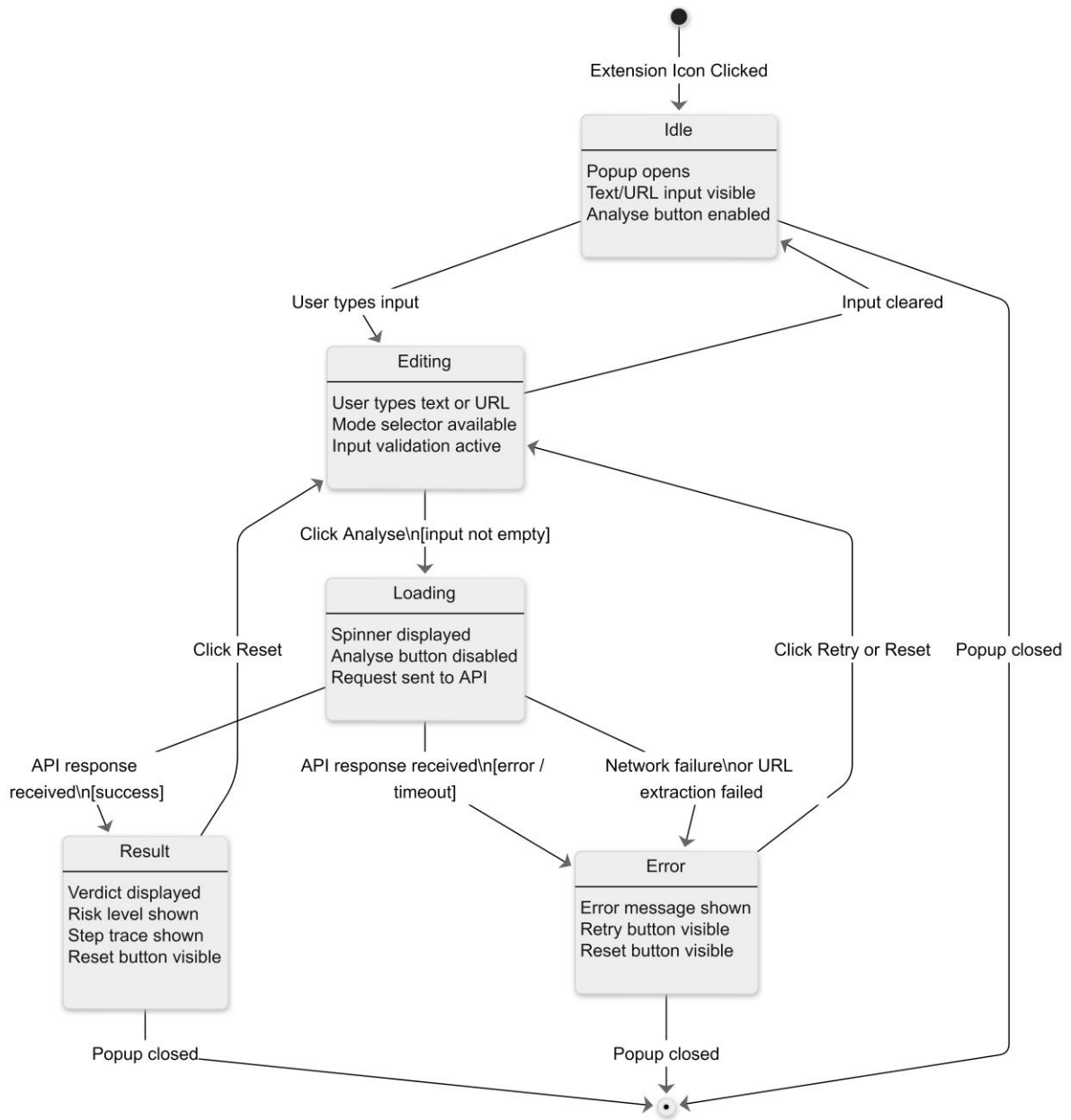


Figure 24: Extension State Diagram (ESD)

Testing

Functional Testing Evidence:

Test Case	Description	Expected Result	Status
TC1	Submit headline text for analysis	Returns verdict with Model A results	Pass
TC2	Submit full article	Returns verdict with	Pass

	text	Model A + Model B results	
TC3	Submit valid news URL	Extracts article content and returns analysis	Pass
TC4	Submit invalid/paywalled URL	Returns UNCERTAIN with FETCH_FAILED error	Pass
TC5	Submit empty text without URL	Returns validation error message	Pass
TC6	Request LIME explanation (force mode)	Returns token importance scores alongside verdict	Pass
TC7	User registration and login flow	Creates accounts, stores session, redirects to homepage	Pass
TC8	Extension popup analysis	Extension sends request, displays verdict in popup	Pass
TC9	Conflicting model predictions	System flags as UNCERTAIN with CONFLICT reason	Pass
TC10	Low confidence prediction	System flags as UNCERTAIN with LOW_CONFIDENCE reason	Pass

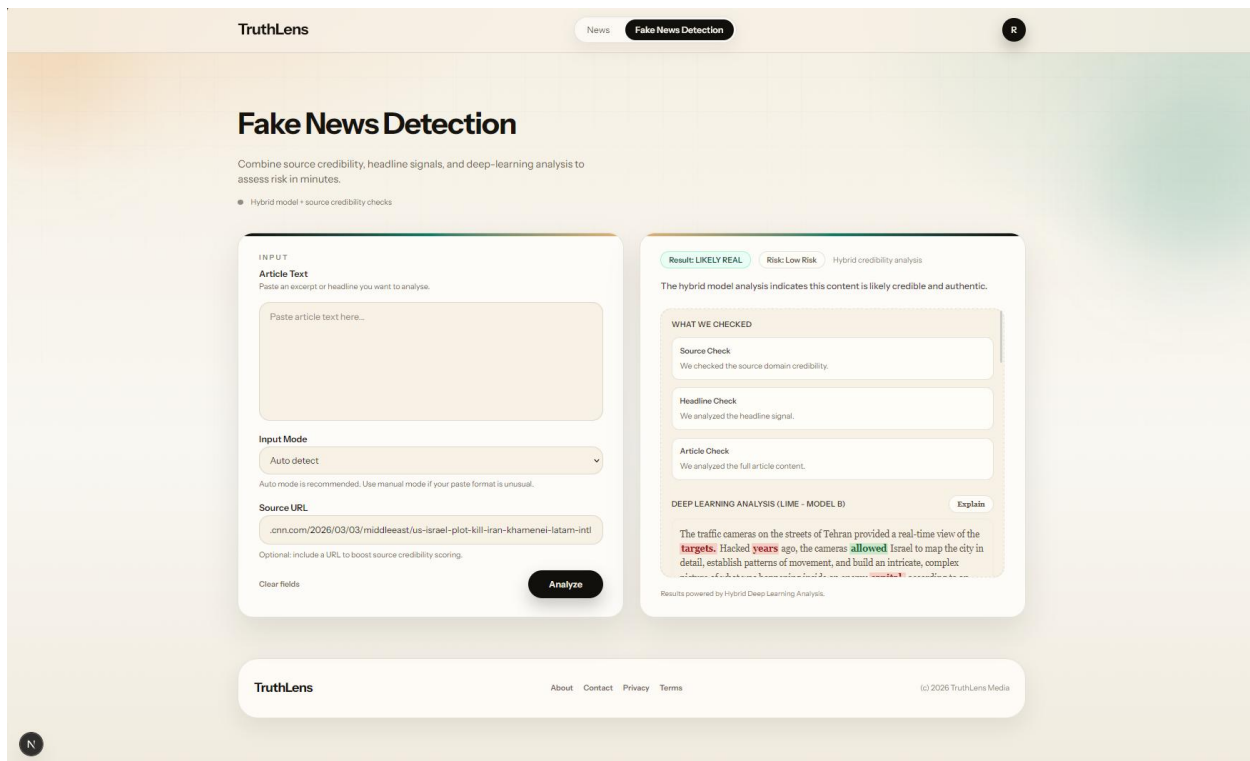


Figure 25: Fake News Detection Testing

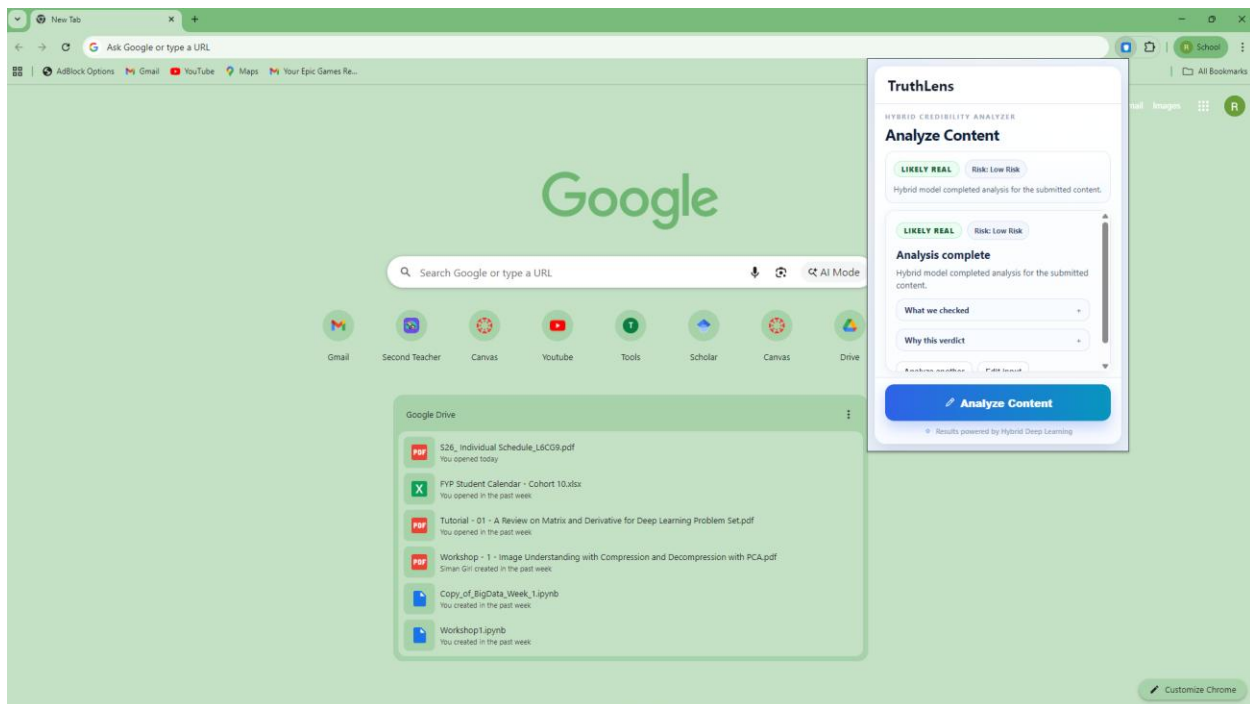


Figure 26: Fake news Detection extension testing

AI-Specific Artefact Documentation

Data Collection

The model training journey involved four distinct phases, each using different datasets and learning from the challenges of the previous phase:

Phase 1 — ISOT Dataset (Failed Approach): The initial training used the ISOT Fake News Dataset containing 44,898 articles (21,417 real from Reuters, 23,481 fake from various unreliable sources). The model achieved 99.7% validation accuracy, which initially appeared excellent. However, diagnostic analysis revealed severe shortcut learning: all real news came from Reuters (professional writing style) while all fake news came from blogs and unreliable sites (casual style). The model had learned to distinguish writing style rather than misinformation content. Additional issues included 6,307 duplicate articles, 931 train/test overlapping articles (data leakage), 687 empty/null texts, complete subject column separation (0% overlap between fake and real subject categories), and massive text artefact differences (Twitter handles: Fake 49.7% vs Real 3.3%; URLs: Fake 17% vs Real 0%; emails: Fake 49.5% vs Real 0.2%). This phase was abandoned, but the diagnostic tools and cleaning pipelines developed (`diagnose_data_leakage.py`, `prepare_kaggle_fixed.py`) became valuable project artefacts.

Phase 2 — LIAR Dataset (Model A Training Data): The LIAR dataset contains 12,836 short political claims from PolitiFact, labelled with fine-grained truthfulness ratings that were mapped to binary REAL/FAKE labels. This dataset was selected because labels are based on factual accuracy verified by professional fact-checkers rather than writing style, addressing the shortcut learning problem. The model achieved approximately 64% accuracy, which is realistic for this challenging task (short claims with limited context). This model became Model A, optimized for headline and claim analysis with a maximum token length of 128.

Phase 3 — FakeNewsNet Dataset: FakeNewsNet provides 23,000+ fact-checked articles from PolitiFact and GossipCop. The model achieved 98.9% test accuracy. While this was a significant improvement, testing revealed limitations with satire detection and well-written fabrications. This led to the decision to combine multiple datasets for improved generalization.

Phase 4 — Diverse Combined Dataset (Model B Training Data): The final training dataset for Model B combined multiple long-article sources including FakeNewsNet, cleaned ISOT/diverse

corpora, and curated synthetic/targeted/new-real article sets. The dataset was split into separate train/validation/test sets with no overlap. In the checked pipeline configuration, COVID ingestion is disabled (enabled: false). Data preparation included text cleaning (removal of URLs, email addresses, Twitter handles, metadata), duplication, minimum text length filtering, and stratified/source-aware splitting.

Dataset cleaning and preprocessing was performed using custom Python scripts including: `prepare_kaggle.py` and `prepare_kaggle_fixed.py` for ISOT data; `prepare_liar.py` for LIAR data; `prepare_fakenewsnet.py` for FakeNewsNet data; and `prepare_diverse_dataset.py` for the combined diverse dataset. Diagnostic tools (`check_data_quality.py`, `diagnose_data_leakage.py`) were developed to systematically identify and resolve data quality issues before training.

Dataset	Source	Split	REAL Samples	FAKE Samples	Total	Purpose in Project
ISOT	University of Victoria — Kaggle	Train/Val/Test	21,417	23,481	44,898	Phase 1 (abandoned — shortcut learning detected)
LIAR	William Yang Wang (2017) — PolitiFact	Train/Val/Test	~6,418	~6,418	12,836	Model A training — headline/claim classification
FakeNewsNet (PolitiFact + GossipCop)	KDD Lab, ASU — GitHub	Train/Val/Test	~4,500	~4,500	~9,000	Phase 3 article training; included in final diverse dataset
COVID-19	Patwa et al.	Train/Val/Test	~3,300	~3,300	~6,600	Methodologi

Fake News	(2021) — GitHub/Kaggle	st			0	cal reference only (not included in final checked Model B training profile)
GitHub Diverse Collection	Multiple curated sources — GitHub	Train/Val/Te st	~3,200	~3,200	~6,40 0	Additional diversity for Model B generalizatio n
Diverse Combined (Final)	FakeNewsNet + cleaned ISOT/diverse + curated article sets	Train/Val/Te st	~11,000	~11,000	~22,0 00	Model B training — article-level classification (final checked profile)
liar_test.csv	Derived from LIAR	Test only	—	—	1,267	Final evaluation of Model A
diverse_test.cs v	Derived from Diverse Combined	Test only	1,682	1,615	3,297	Final evaluation of Model B

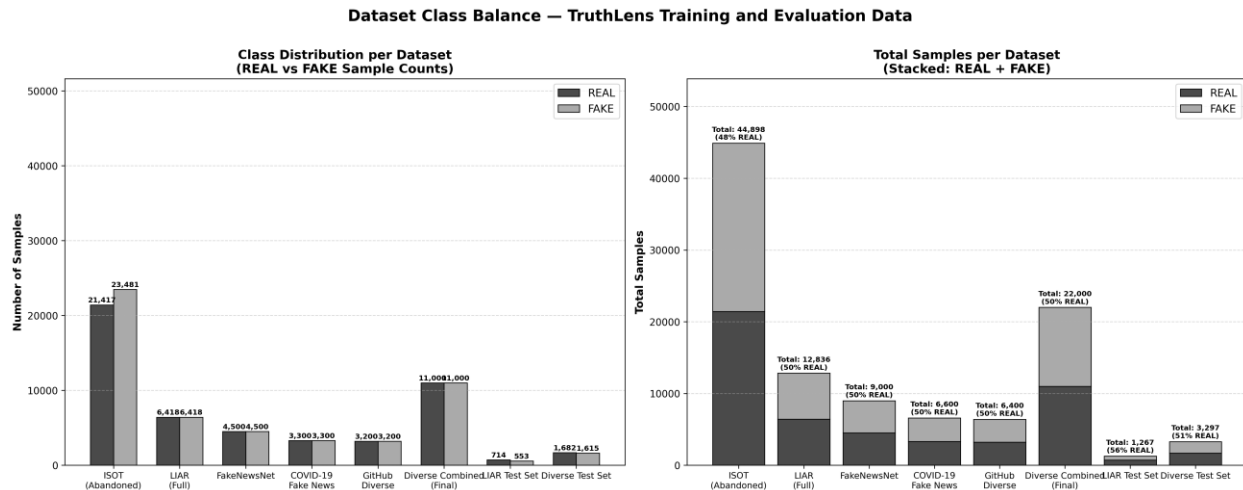


Figure 27: Dataset Class Balance

Model Development

Both Model A and Model B share the same base architecture—RoBERTa-base fine-tuned with LoRA—but differ in their training data, maximum input length, and deployment context within the hybrid pipeline.

Model A (Headline/Claim Classifier): Trained on a source-disjoint short-text corpus anchored by LIAR and augmented with cleaned ISOT and curated headline sets. Configuration (final pipeline/deployed adapter): RoBERTa-base backbone, LoRA rank $r=16$, $\alpha=32$, dropout=0.05, target modules=['query', 'value'], maximum token length=128, learning rate= $2e-4$, and up to 5 epochs with early stopping (patience=2). The model is optimised for short text inputs (headlines, claims, social posts).

Model B (Article Classifier): Trained on a multi-source long-article corpus (FakeNewsNet + cleaned diverse/ISOT + curated article sets). Configuration (final deployed run): RoBERTa-base backbone, LoRA rank $r=16$, $\alpha=32$, dropout=0.05, adapter target modules including query/value/key, maximum token length=512, learning rate= $1e-4$, and 3 training epochs in the final profile. The model is optimized for full-article analysis and longer-context reasoning.

Training was performed using the Hugging Face Trainer API with PyTorch as the backend framework. Mixed-precision training (FP16) was enabled when GPU was available to reduce memory usage and accelerate training. The DataCollatorWithPadding was used for efficient

dynamic batching. Both models were saved as LoRA adapter weights alongside the tokenizer, enabling lightweight deployment without distributing the full base model weights.

Parameter	Model A (Headline)	Model B (Article)
Base Model	RoBERTa-base (125M params)	RoBERTa-base (125M params)
Fine-tuning Method	LoRA (r=16, alpha=32)	LoRA (r=16, alpha=32)
Training Dataset	LIAR-anchored + cleaned ISOT + curated headline sets	FakeNewsNet + cleaned ISOT/diverse + curated article sets
Max Token Length	128	512
Dropout	0.05 (deployed adapter)	0.05 (deployed adapter)
Learning Rate	2e-4	1e-4
Training Epochs	5 (with early stopping)	3 (final run profile)
Batch Size	16	8
Gradient Accumulation	2 steps	4 steps
Optimiser	AdamW (weight decay=0.01)	AdamW/Focal-loss setup (weight decay=0.01)
Warmup	0.06	0.06
Best Model Selection	Macro F1	Fake recall
Trainable Parameters	~1% of full model	~1% of full model

Optimisation and Evaluation

Overfitting Prevention: Through the iterative training journey, multiple regularization strategies were tested, including higher dropout experiments (0.1, 0.15, 0.2, 0.3), learning-rate tuning, early stopping, label smoothing, and weight decay adjustments. The final deployed adapters use LoRA dropout of 0.05, with uncertainty handling and confidence thresholds in inference to reduce overconfident outputs.

Data Leakage Mitigation: Custom diagnostic tools were developed to detect and eliminate data leakage. The `diagnose_data_leakage.py` script checks for metadata leakage (title/subject/date appearing in text), subject distribution analysis, and text artefact distribution (URLs, Twitter

handles, emails). The `fix_data_issues.py` and `prepare_kaggle_fixed.py` scripts remove identified leakage sources. Train/test/validation splits were verified for zero overlap.

Pipeline V2 Advanced Measures: A more sophisticated training pipeline (`pipeline_v2/`) was developed with additional anti-shortcut measures including: entity masking (replacing named entities with generic tokens to prevent person-label correlations), artefact token dropout (randomly removing tokens predictive of dataset origin rather than content, such as 'subscribe' and 'newsletter'), source-aware splitting (keeping entire news sources in either train or test, never both), and Leave-One-Dataset-Out (LODO) evaluation for rigorous cross-dataset generalisation testing.

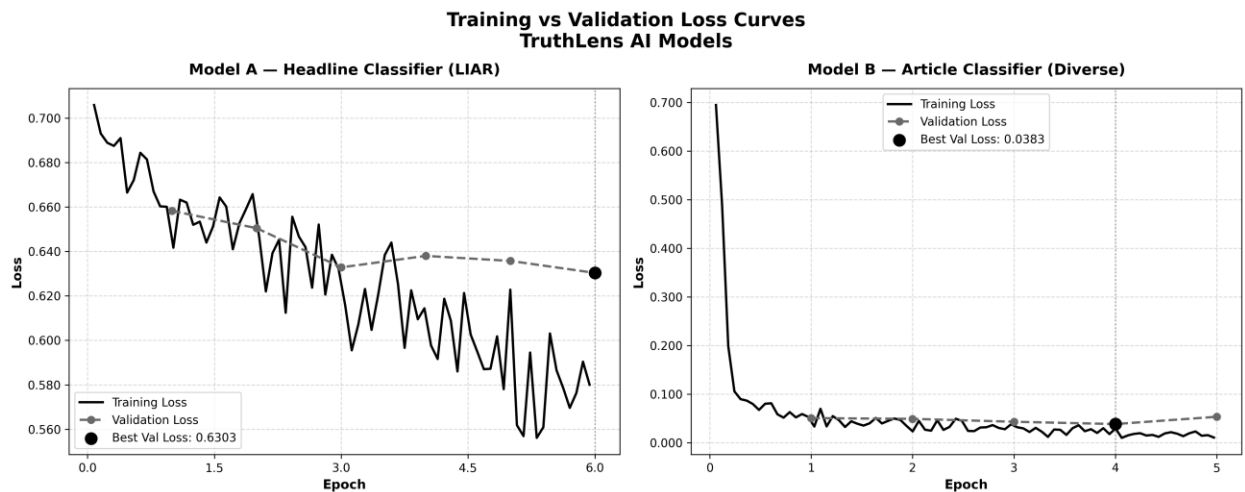


Figure 28: Training vs Validation Loss Curve

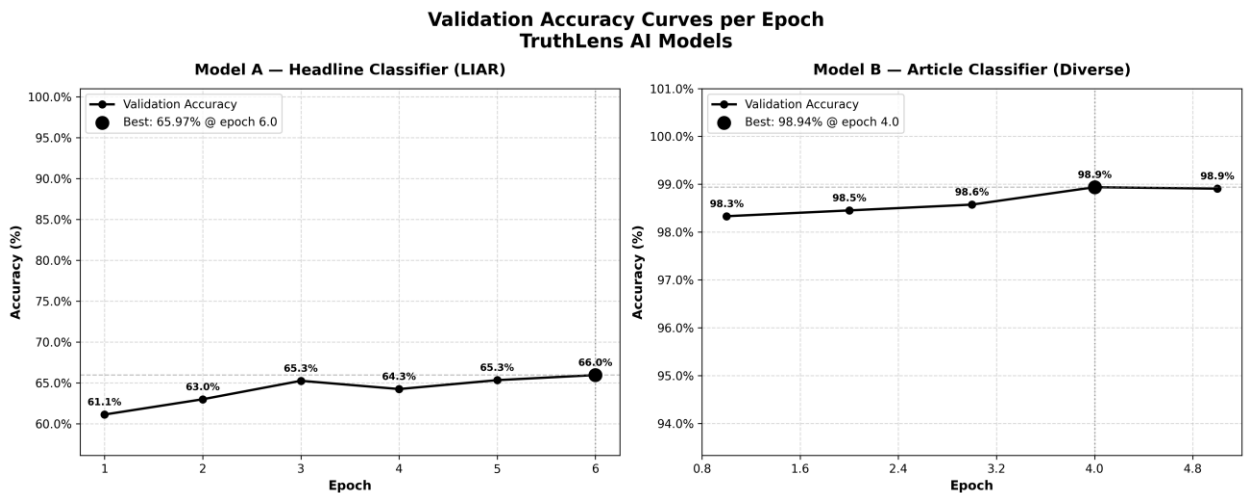


Figure 29: Validation Accuracy Curves per Epoch

AI Integration into the Application

The AI models are integrated into the application through a RESTful API architecture. The FastAPI backend exposes a single primary endpoint: POST /predict, which accepts a JSON payload containing: text (string—the article text or headline to analyse), url (optional string—a source URL for domain credibility checking and optional article extraction), input_mode (enum: 'auto', 'headline_only', 'full_article', 'headline_plus_article'), and explanation_mode (enum: 'none', 'auto', 'force').

The backend's HybridModelLoader class manages the lifecycle of both models. On startup, it loads the LoRA adapter weights for Model A and Model B, initialises the source credibility database from a JSON file, and creates the LIME text explainer. The InferenceModel class handles tokenization, forward pass, softmax probability computation, and label mapping for each individual model. Inference is performed with PyTorch's @torch.inference_mode() decorator to disable gradient computation and reduce memory usage.

The analysis pipeline follows this sequence: (1) Source Check—the URL domain is matched against the source credibility database, assigning scores: +3 for known fake/satire domains, +1 for biased, -2 for known reliable. (2) Input Parsing—the HybridModelLoader.parse_input() method analyses the input text structure, detecting whether it contains a headline, article body, or both. It applies dateline stripping (removing Reuters/AP/AFP datelines), delimiter detection, and headline plausibility checks. (3) Model Routing—based on the detected input structure and selected input_mode, the system decides which models to run (Model A for headlines with >0 words, Model B for article bodies with ≥80 words). (4) Model Inference—each selected model produces a label and confidence score. Score impacts are assigned: Model A FAKE (conf>0.60) = +2, REAL (conf>0.60) = -2; Model B FAKE (conf>0.85) = +1, REAL (conf>0.97) = -1. (5) Conflict Resolution—if both models run and their predictions disagree with high confidence (both >80%), the verdict is overridden to UNCERTAIN. (6) Verdict Generation—total score ≥2 = SUSPICIOUS (High Risk), ≤-2 = LIKELY REAL (Low Risk), else = UNCERTAIN (Needs Review). (7) LIME Explanation—if requested, LIME generates token-level feature importance by perturbing the input text and observing prediction changes.

The Next.js frontend communicates with the backend through an API route proxy (/api/predict), which forwards requests to the FastAPI backend. This architecture centralises CORS

management and allows the frontend to add authentication headers or rate limiting as needed. The browser extension uses the same web API contract and targets the frontend endpoint by default (localhost:3000), with runtime override options in extension storage.

Sample API Request:

```
{ "text": "Breaking: Scientists discover cure for all diseases", "url": "", "input_mode": "auto",  
  "explanation_mode": "auto" }
```

Sample API Response (summarised):

```
{ "final_score": 2, "verdict": "SUSPICIOUS", "risk_level": "High Risk", "steps": [...],  
  "model_outputs": { "model_a": { "ran": true, "label": "FAKE", "confidence": 0.89 }, "model_b":  
    { "ran": false } }, "article_class": "FAKE" }
```

Comparing Algorithm Performance (Model A vs Model B vs Hybrid)

Model A vs Model B Performance Comparison:

Model A (Headlines): Batch test on 101 curated samples—Accuracy: 94.06%, Macro F1: 94.03%. Strengths: fast inference on short text, good at detecting misleading political claims. Weaknesses: limited context from short inputs, lower confidence in ambiguous headlines. No REAL articles were misclassified as FAKE; 6 FAKE items were missing (predicted REAL).

Model B (Articles): Batch test on 200 curated samples—Accuracy: 97.50%, Macro F1: 97.04%. Strengths: captures broader context and argumentation patterns, higher overall accuracy. Weaknesses: requires minimum 80 words for reliable analysis, slightly higher computational cost due to 512-token input. 2 REAL articles were misclassified as FAKE; 3 FAKE articles were missing.

Hybrid System (Combined): The three-layer hybrid approach compensates for individual model weaknesses. When Model A and Model B disagree with high confidence, the system flags the result as UNCERTAIN rather than forcing an incorrect binary prediction. Source credibility checks provide an additional signal independent of text content, catching known unreliable domains that might produce well-written but fabricated content.

Metric	Model A (Headlines)	Model B (Articles)	Hybrid System
Accuracy	94.06%	97.50%	Combined 3-layer approach
Macro F1	94.03%	97.04%	With conflict resolution
False Positives (REAL→FAKE)	0	2	Reduced by conflict detection
False Negatives (FAKE→REAL)	6	3	Reduced by hybrid scoring
Input Length	Short (≤ 128 tokens)	Long (≤ 512 tokens)	Auto-detected routing
Primary Strength	Political claims	Article content patterns	Multi-signal robustness

AI Testing — Accuracy Visualisations

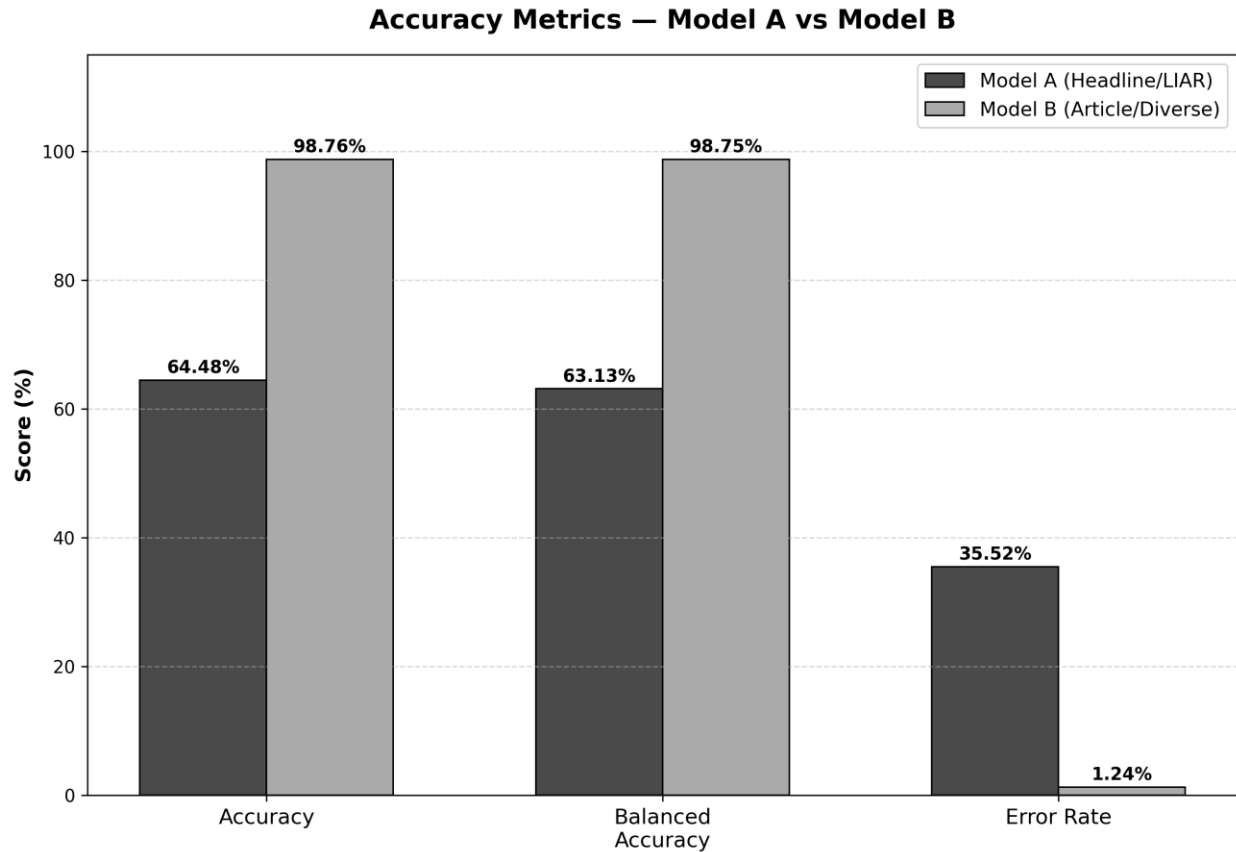


Figure 30: Accuracy Metrics

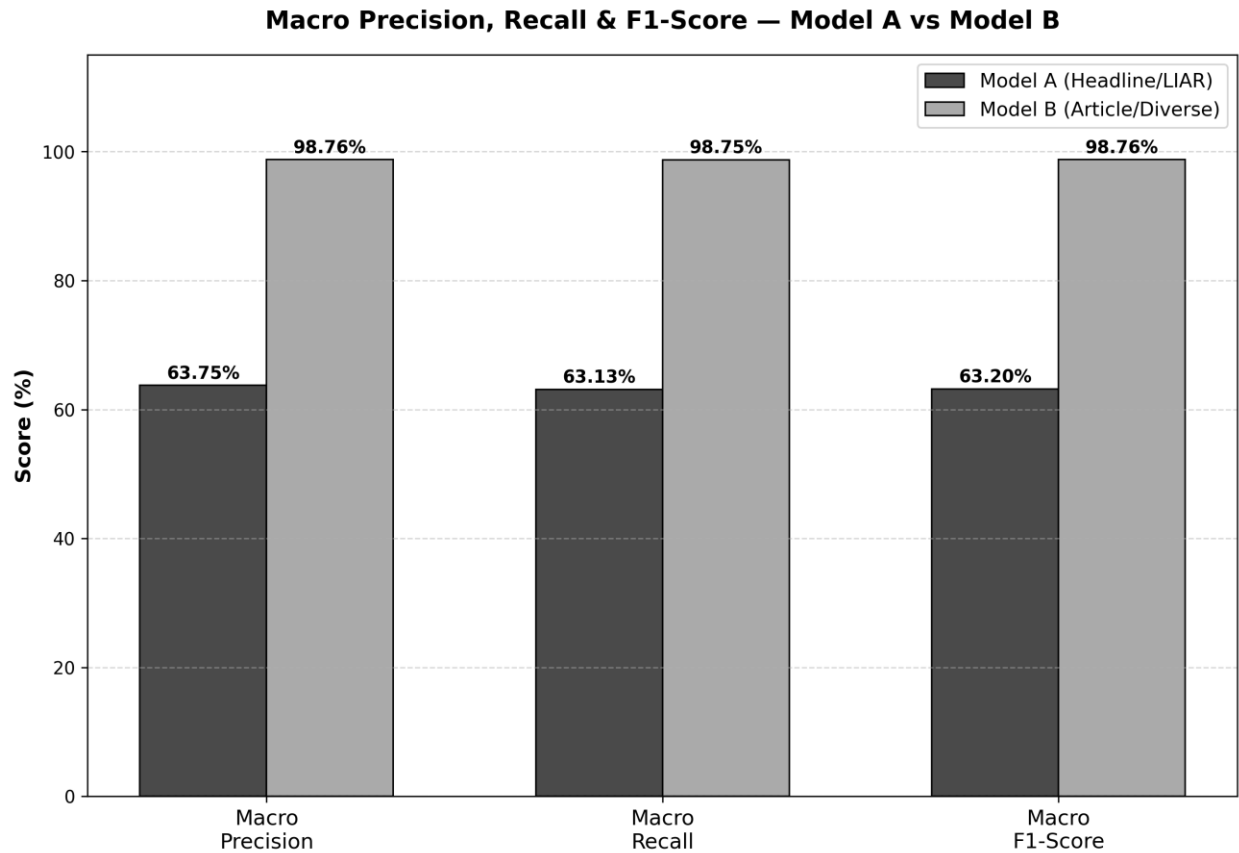


Figure 31: macro Precision, Recall and F1-Score

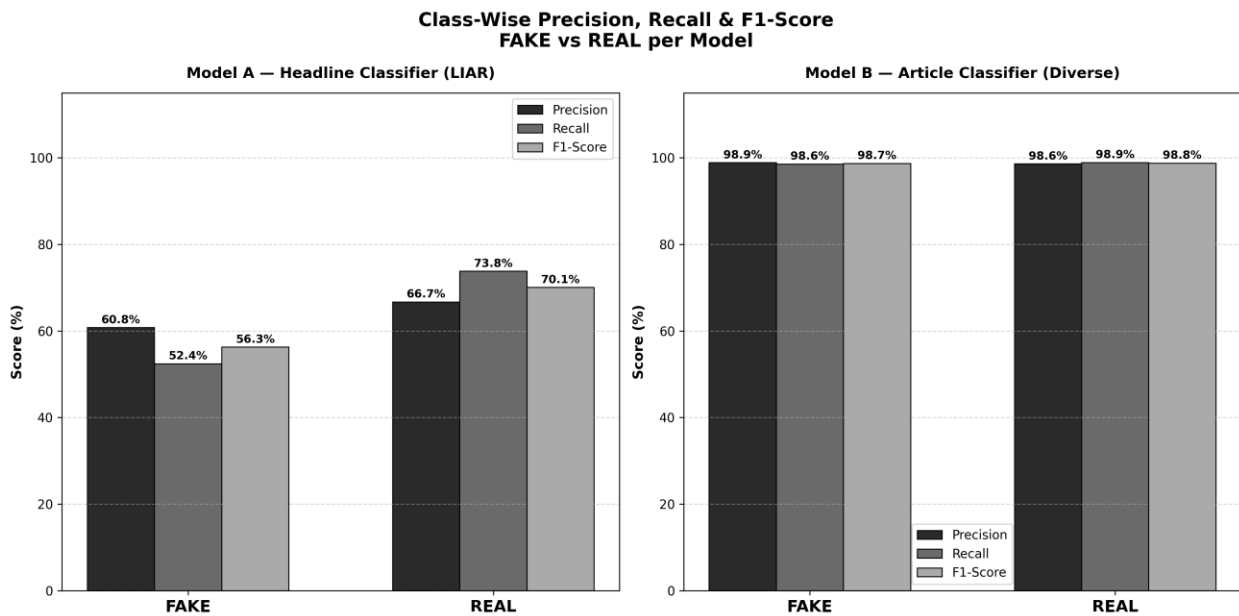


Figure 32: Class-Wise Precision, Recall and F1-Score

Confusion Matrix

Detailed confusion matrices for both models are presented below. These matrices show the distribution of correct and incorrect predictions across both classes:

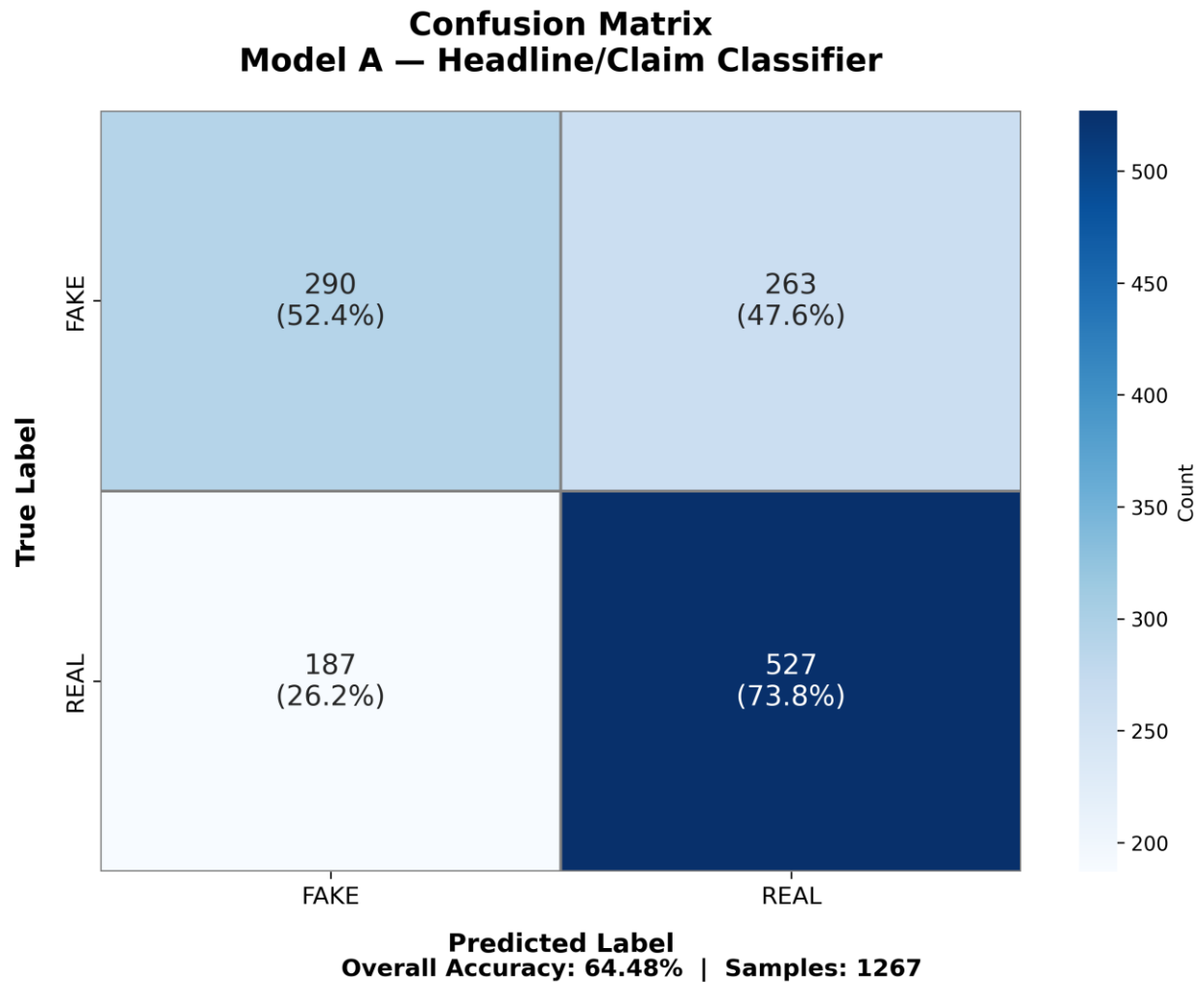


Figure 33: Confusion Matrix (Model A)

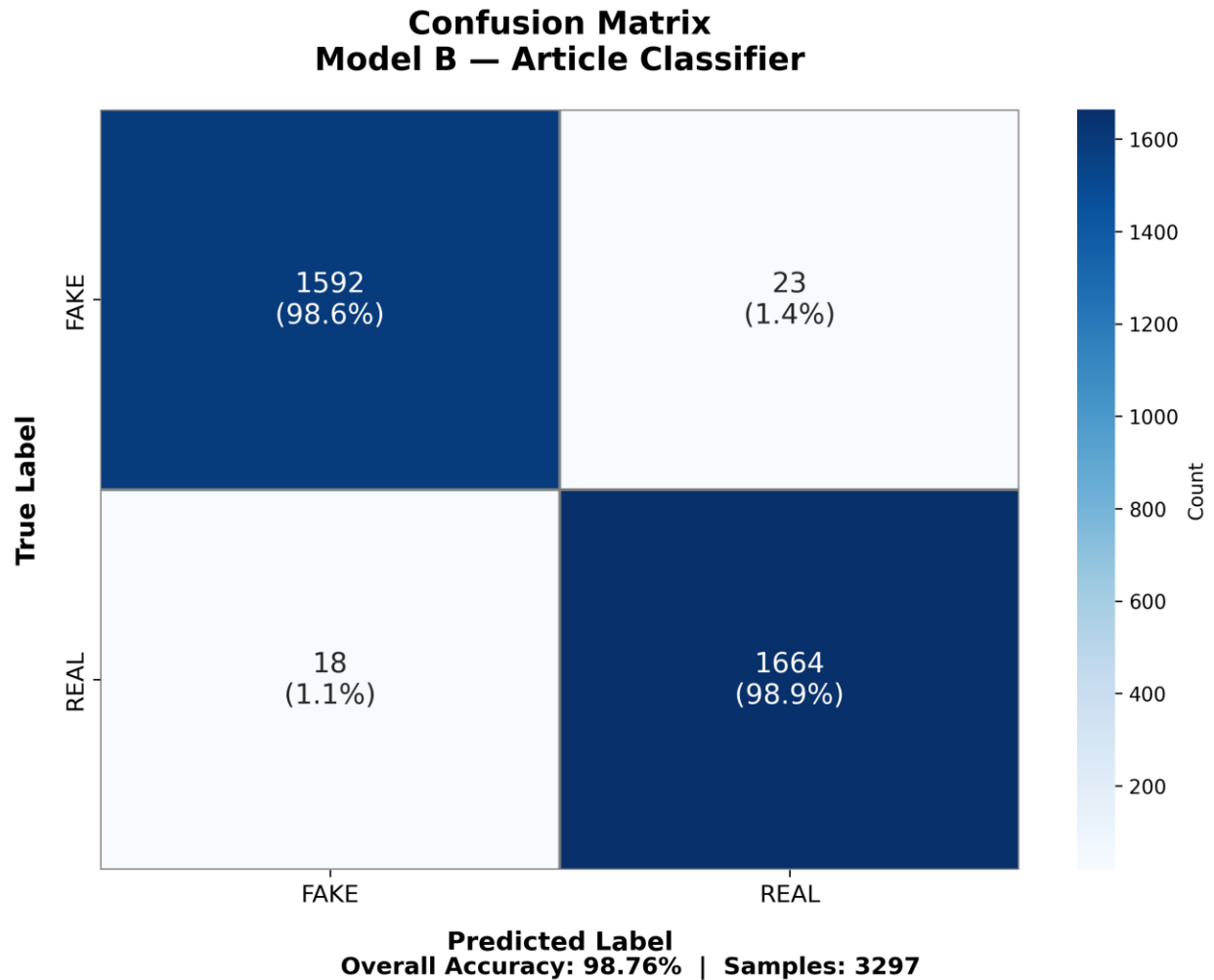


Figure 34: Confusion Matrix (Model B)

ROC Curve

The Receiver Operating Characteristic (ROC) curves for both models plot the True Positive Rate against the False Positive Rate at various classification thresholds, with the Area Under the Curve (AUC) providing a threshold-independent measure of classification quality:

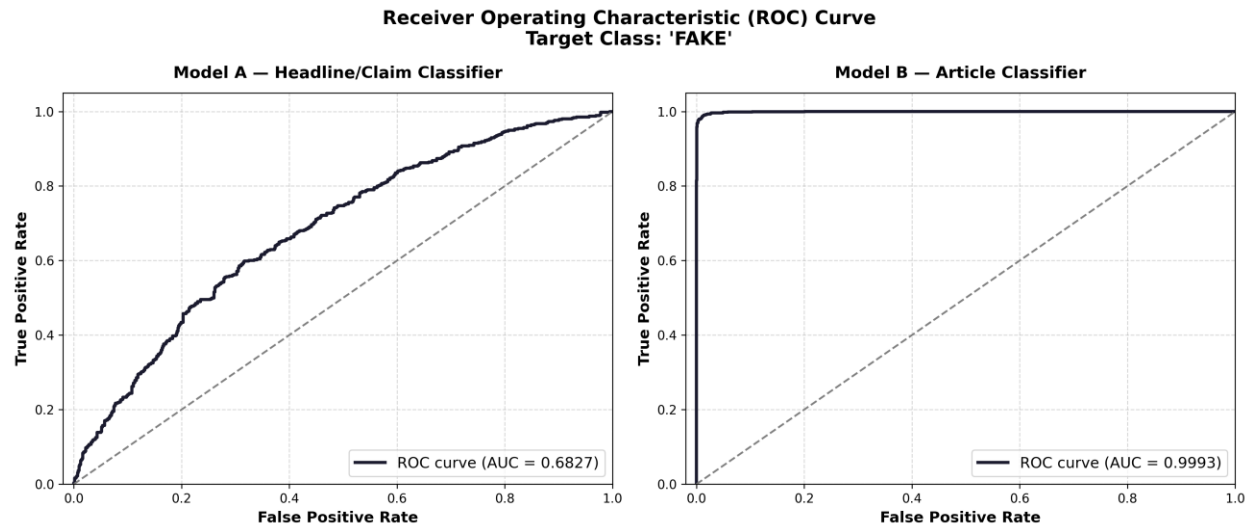


Figure 35: ROC Curve

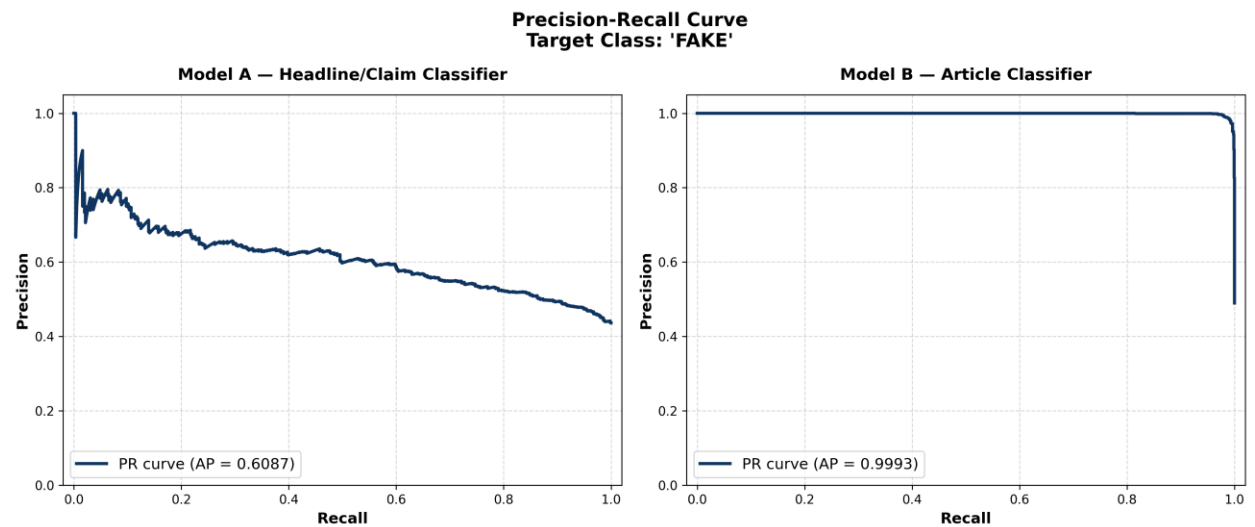


Figure 36: Precision-Recall Curve

Conclusion

This project set out to design, implement, and evaluate an AI-powered fake news verification platform—TruthLens—that provides timely and interpretable credibility assessments for English-language news content. Reflecting on the aims and objectives, the following conclusions can be drawn:

Objective 1 (Web-based verification platform): Successfully delivered. TruthLens provides a fully functional Next.js web application with a dedicated fake news detection page, news aggregation with credibility signals, and user account management. The platform integrates two transformer-based models covering both short-form and long-form text analysis.

Objective 2 (Multiple input methods): Successfully delivered. The system supports manual text entry (headlines, claims, or full articles) and URL-based input with automatic article extraction via *trafilatura*. The input parsing system automatically detects input structure and routes to the appropriate model(s).

Objective 3 (Real-time classification with uncertainty): Successfully delivered. The hybrid pipeline produces verdicts (Suspicious, Uncertain, Likely Real), risk levels, step-by-step analysis traces, and explicit uncertainty indicators for low-confidence predictions and model disagreements.

Objective 4 (Explainability via LIME): Successfully delivered. LIME integration provides token-level feature importance highlighting, supported in both automatic and force modes. Stopword filtering ensures explanation quality.

Objective 5 (Browser extension): Successfully delivered. The Chrome extension (Manifest V3) provides a popup-based verification interface using the same core API contract as the web app, with a compact UX tailored for extension constraints.

Objective 6 (Quantitative evaluation): Successfully delivered. Batch tests with curated datasets produced Model A accuracy of 94.06% (F1 = 94.03%) and Model B accuracy of 97.50% (F1 = 97.04%). These batch metrics are from curated held-out evaluation sets (Model A: n=101 headlines; Model B: n=200 articles). They are distinct from training-validation metrics recorded during model development, so the values are not directly comparable.

Addressing the Academic Question: The project demonstrates that an effective architecture for web-based fake news detection combines: (a) a modular hybrid pipeline with multiple complementary detection layers rather than a single monolithic model; (b) parameter-efficient fine-tuning (LoRA) to enable transformer deployment without prohibitive computational requirements; (c) explicit uncertainty handling to prevent overconfident predictions and maintain user trust; and (d) post-hoc explainability (LIME) to provide transparency into model reasoning. The three-layer architecture (source credibility, headline analysis, article analysis) with conflict resolution represents a practical instantiation of this ideal architecture, balancing detection capability with honest uncertainty communication.

Key discoveries from the development process include: (1) dataset quality is at least as important as model architecture—the 99.7% accuracy achieved on the ISOT dataset was entirely artifactual, caused by shortcut learning and data leakage; (2) hybrid approaches that combine multiple signals are more robust than single-model systems; (3) explicit uncertainty handling (conflict detection, low-confidence flagging) is essential for maintaining user trust in a decision-support context; and (4) real-world deployment introduces challenges (URL extraction failures, diverse writing styles, domain shift) that controlled benchmark evaluations do not capture.

Critical Evaluation of the Project

Report Evaluation: This report documents the complete development of the lifecycle of TruthLens, from initial research through iterative model development, system implementation, and evaluation. A particular strength is the honest documentation of failed approaches (the ISOT dataset experience) alongside successful outcomes, providing a realistic picture of applied AI/ML development rather than presenting only positive results. The literature review covers a broad range of relevant research, though deeper engagement with specific mathematical formulations in some papers could strengthen the theoretical grounding.

System Evaluation: The TruthLens platform successfully delivers on its core promise of providing AI-powered credibility assessments through a user-friendly interface. Key strengths include: the hybrid three-layer architecture that provides defense in depth; the explicit uncertainty handling that prevents overconfident false predictions; the LIME explainability module that improves transparency; and the comprehensive input parsing system that handles diverse text formats. Limitations include: the system is evaluated on curated batch tests rather than large-scale real-world deployment; URL extraction depends on website accessibility and may fail for paywalled or dynamically rendered content; the source credibility database requires manual maintenance; and model performance may degrade under significant domain shift from the training data distribution.

Process Evaluation: The Agile methodology proved appropriate for this project, enabling rapid iteration when challenges were encountered (particularly during the model training journey). Regular supervisor meetings provided valuable feedback that guided development priorities. The use of Git for version control and structured documentation (README files, journey logs, fix guides) facilitated systematic progress tracking. Areas for improvement include earlier integration of automated testing, more structured sprint planning documentation, and parallel development of the evaluation framework alongside the system implementation rather than sequentially.

Planning and Management: The 47-week timeline was broadly appropriate, though the initial underestimation of data quality consumed additional time that could have been allocated to more extensive evaluation. The logsheets document consistent weekly progress and supervisor

engagement. The Gantt chart was maintained with major milestones, though more granular task tracking could improve future project management.

Quality of Sources: The literature review draws from peer-reviewed conference and journal papers spanning 2018-2024, providing current coverage of the rapidly evolving fake news detection field. The inclusion of both text-focused and multimodal studies provides breadth, while the specific relevance of each paper to TruthLens design decisions is clearly articulated. Additional sources from the grey literature (documentation, blog posts, library READMEs) were used for technical implementation details.

Self-Reflection

This project has been a transformative learning experience across multiple dimensions. Technically, I gained deep practical knowledge of transformer-based NLP, from understanding the mathematical foundations of attention mechanisms to the practical realities of fine-tuning with limited computational resources. The iterative model training journey, particularly the experience of achieving 99.7% accuracy only to discover it was entirely due to dataset artefacts—was one of the most valuable lessons of the entire project. It taught me that impressive metrics are meaningless without rigorous validation, and that understanding your data is at least as important as choosing the right model architecture.

From a software engineering perspective, I developed skills in full-stack development spanning Python backend (FastAPI), TypeScript frontend (Next.js/React), database management (MongoDB), and browser extension development (Chrome Manifest V3). The experience of integrating ML models into a production-like web application highlighted challenges that are not covered in typical ML coursework—such as API latency management, error handling for real-world input variability, and the importance of explicit uncertainty communication.

Professionally, this project improved my project management skills through the practice of iterative planning, regular progress tracking (logsheets), and the discipline of documented decision-making. Working on an end-to-end system from research through deployment gave me confidence in tackling complex software projects independently. The experience of writing diagnostic tools, cleaning pipelines, and comprehensive documentation will be directly applicable in industry settings

If I were to undertake this project again, I would: (a) invest more time in dataset analysis before beginning any model training; (b) implement automated testing earlier in the development process; (c) explore additional evaluation methodologies such as user studies or A/B testing with real users; and (d) consider a more modular microservices architecture to facilitate future extensions such as multilingual support.

Evidence of Project Management

Project management was carried out through a combination of weekly supervision, milestone tracking, and version-controlled development workflow. Progress was reviewed in regular supervisor meetings and documented in signed log sheets, while major milestones were tracked against the project Gantt chart (planning, model development, backend integration, frontend/extension integration, testing, and report writing).

To improve delivery control, GitHub was used as the central project management and development platform. The repository (<https://github.com/Rigen-Manandhar/TruthLens-Fake-News-Detection>) was used for source control, incremental delivery, rollback safety, and evidence of progress. Work was organized into feature/fix branches, then merged into the main branch after testing. Commit history was used as a development audit trail, linking technical changes to project milestones and report deliverables. GitHub also supported issue tracking for bug fixing and task planning, including items such as data leakage fixes, model tuning iterations, API integration, and UI refinements.

This workflow improved transparency, reduced integration risk, and made it easier to monitor project velocity throughout the development lifecycle. It also ensured that project artefacts (code, scripts, model configuration updates, and documentation) remained reproducible and traceable to specific stages of the final-year project timeline. The proof of commits is given below:

Rigen-Manandhar / TruthLens-Fake-News-Detection

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security

Insights

Settings

Type to search

Commits

main

All users

All time

Commits on Feb 28, 2026

Frontend design changes and improvements

Rigen-Manandhar committed 5 days ago

813089b

Implemented the news confidence showcase in the news page and some frontend design and color changes

Rigen-Manandhar committed 5 days ago

86b332d

Commits on Feb 16, 2026

Modifies LIME to stop showing stop words in the explanation

Rigen-Manandhar committed 2 weeks ago

08041c9

feat: Implement core fake news detection system with backend model inference, text processing, and a frontend user interface.

Rigen-Manandhar committed 2 weeks ago

8cd7d9a

Tweaked the Model Loader so it used the 2 new models instead of the previous models

Rigen-Manandhar committed 2 weeks ago

894274a

Imported New Improved Model A and B replacing the older models

Rigen-Manandhar committed 2 weeks ago

a88a544

Commits on Feb 4, 2026

Refactor ConfirmDialog component: remove unused state and simplify open condition

Rigen-Manandhar committed last month

886bd6b

Implement user profile management and password update features; add ConfirmDialog component for logout confirmation

Rigen-Manandhar committed last month

06e4298

Improved the Fake news detection page frontend design and added code so the user can only access the feature when they are logged in

Rigen-Manandhar committed last month

ec9e922

Improved the frontend design of the news page

Rigen-Manandhar committed last month

0542c9b

Improved the frontend design for Login and Signup page, added google signup and login on top of the regular Signup and login

Rigen-Manandhar committed last month

b148b35

Improved the frontend design for Login and Signup page, added google signup and login on top of the regular Signup and login

Rigen-Manandhar committed last month

b148b35

Commits on Jan 31, 2026

Fixed warnings and errors in the code

Rigen-Manandhar committed on Jan 31

eac5c60

Changed some file names

Rigen-Manandhar committed on Jan 31

0b1d2a6

Added the database (MongoDB) to store the user info, passwords are hashed and successfully stored. implemented and updated states for user logged in and logged out.

Rigen-Manandhar committed on Jan 31

a73246f

Commits on Jan 27, 2026

BriefUsed 2 finetuned roberta models to make a hybrid model to detect news better. Backend: Used a finetuned roberta model trained in only LIAR dataset while another roberta model lora-diverse tra...

Rigen-Manandhar committed on Jan 27

ec1280

Commits on Jan 19, 2026

Added Backend and Frontend in one repo for ease and quality of life, as well as Added LORA for highlights of text that motivate the model to predict Real and Fake

Rigen-Manandhar committed on Jan 19

1692bb1

Commits on Jan 18, 2026

Small git changes

Rigen-Manandhar committed on Jan 18

acc4330

Commits on Jan 5, 2026

Small Changes in file paths

Rigen-Manandhar committed on Jan 5

a1d0c4b

Commits on Nov 22, 2025

Update background gradient class to linear in Home and FakeDetection pages for consistent styling

Rigen-Manandhar committed on Nov 22, 2025

253ccf6

Refactor NewsCard component: update class names for better responsiveness and add eslint directive for image element

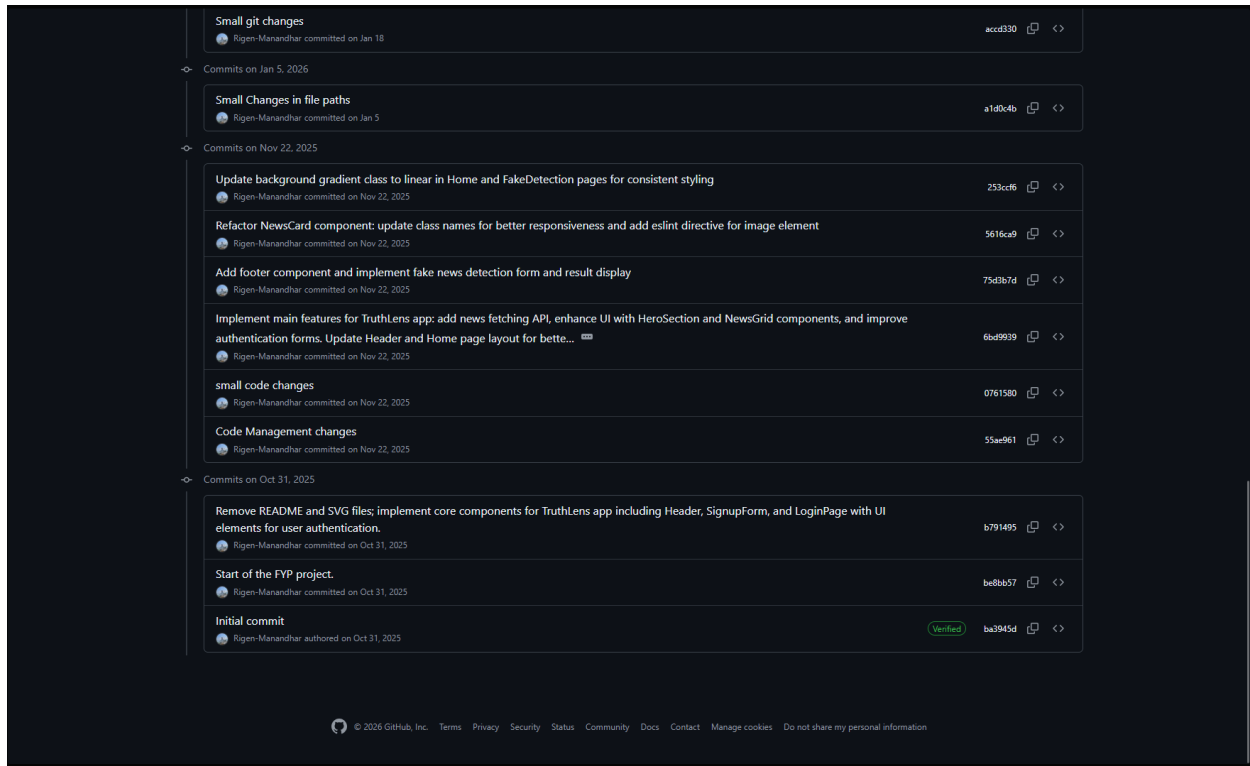
Rigen-Manandhar committed on Nov 22, 2025

5616ca9

Add footer component and implement fake news detection form and result display

Rigen-Manandhar committed on Nov 22, 2025

75d3b7d



Log Sheets — Signed and Scanned by Supervisor

The following logsheets document weekly progress meetings with the supervisor throughout the project duration:

PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mrs. Prakriti Regmi
Project Title: Fake News Detection	Month: November
What have you done since the last meeting	
Designed Home / Landing page in Figma.	
What do you aim to complete before the next meeting	
Integrate news api and finish the landing page and research about extensions as well as look into the datasets.	
Supervisor comments	

We confirm that the information given in this form is true, complete and accurate.

Student Signature:  _____

Date: November 17th _____

Supervisor Signature:  _____

Date: _____

PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mrs. Prakriti Regmi
Project Title: Fake News Detection	Month: November
What have you done since the last meeting	
Integrated news api and finished the landing page as well as Most of the Frontend design in code, found and researched on datasets too.	
What do you aim to complete before the next meeting	
Start Working on RoBerta model with the fake news dataset.	
Supervisor comments	

We confirm that the information given in this form is true, complete and accurate.

Student Signature:  _____

Date: November 24

Supervisor Signature:  _____

Date: November 24

PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mrs. Prakriti Regmi
Project Title: Fake News Detection	Month: December
What have you done since the last meeting	
Researched about ML and model training and if model is suitable to be Trained in device or google colab. Also set up the venv for testing.	
What do you aim to complete before the next meeting	
Start training program for RoBerta model and use 2 datasets to train.	
Supervisor comments	

We confirm that the information given in this form is true, complete and accurate.

Student Signature: 

Date: December 7th

Supervisor Signature: 

Date:

Faculty of Science and Engineering
School of Mathematics and Computer Science



PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mr. Prakriti Regmi
Project Title: Fake News Detection	Month: December
What have you done since the last meeting	
Trained the Roberta Model in the kaggle Fake news Dataset using locally using Jova To fit the model in the vram. and also made performance metrics.	
What do you aim to complete before the next meeting	
Integrate the Backend and Frontend	
Supervisor comments	
Have prepared with some test cases for the news classifier model test	

We confirm that the information given in this form is true, complete and accurate.

Student Signature: Rigen

Date: 21st Dec

Supervisor Signature: Prakriti Regmi

Date: 21st Dec

We confirm that the information given in this form is true, complete and accurate.

Date: January 11

Date: January 11

PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mrs. Prakriti Regmi
Project Title: Fake News Detection	Month: January
What have you done since the last meeting	
Integrated LIME to display what motivates the AI model to choose Fake Or True and showed it in the frontend.	
What do you aim to complete before the next meeting	
Clean the IOST Dataset and try to retrain the AI model, Use LIAR dataset too to train the model in a shorter form of dataset and if none of those dataset turns out good results focus on finding better datasets instead.	
Supervisor comments	
Ensure completion of the model completion with improved accuracy.	

We confirm that the information given in this form is true, complete and accurate.

Student Signature:  _____

Date: January 25

Supervisor Signature:  _____

Date: January 25

We confirm that the information given in this form is true, complete and accurate.

Date: February 1st

Date: February 1st

PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mrs. Prakriti Regmi
Project Title: Fake News Detection	Month: February
What have you done since the last meeting	
Trained Model A (RoBERTa) on 4 different datasets of only headlines using heavy Preprocessing to make the model actually learn, accuracy at 88% from 99% Suggesting that the model actually learned.	
What do you aim to complete before the next meeting	
Train Model B (RoBERTa) on 3 to 4 datasets but on larger and longer articles rather than headlines using the same preprocessing.	
Supervisor comments	
Update the preprocessing, source contemporary data and re train the model	

We confirm that the information given in this form is true, complete and accurate.

Student Signature:  _____

Date: February 8th

Supervisor Signature:  _____

Date: February 8th

PROJECT MANAGEMENT LOG	
First Name: Rigen	Surname: Manandhar
Student Number: 2408621	Supervisor: Mrs. Prakriti Regmi
Project Title: Fake News Detection	Month: February
What have you done since the last meeting	
Retrained Model A and B on more dataset by scraping Gnews through Newspaper4K Improved accuracy of both the model quite a bit as well	
What do you aim to complete before the next meeting	
Integrate model analysis for the news displayed for the homepage, using both model A and model B for the prediction, store the predicted news in the database for loading times.	
Supervisor comments	
Work on analysing the models performance	

We confirm that the information given in this form is true, complete and accurate.

Student Signature:  _____

Date: February 22nd

Supervisor Signature:  _____

Date: February 22nd

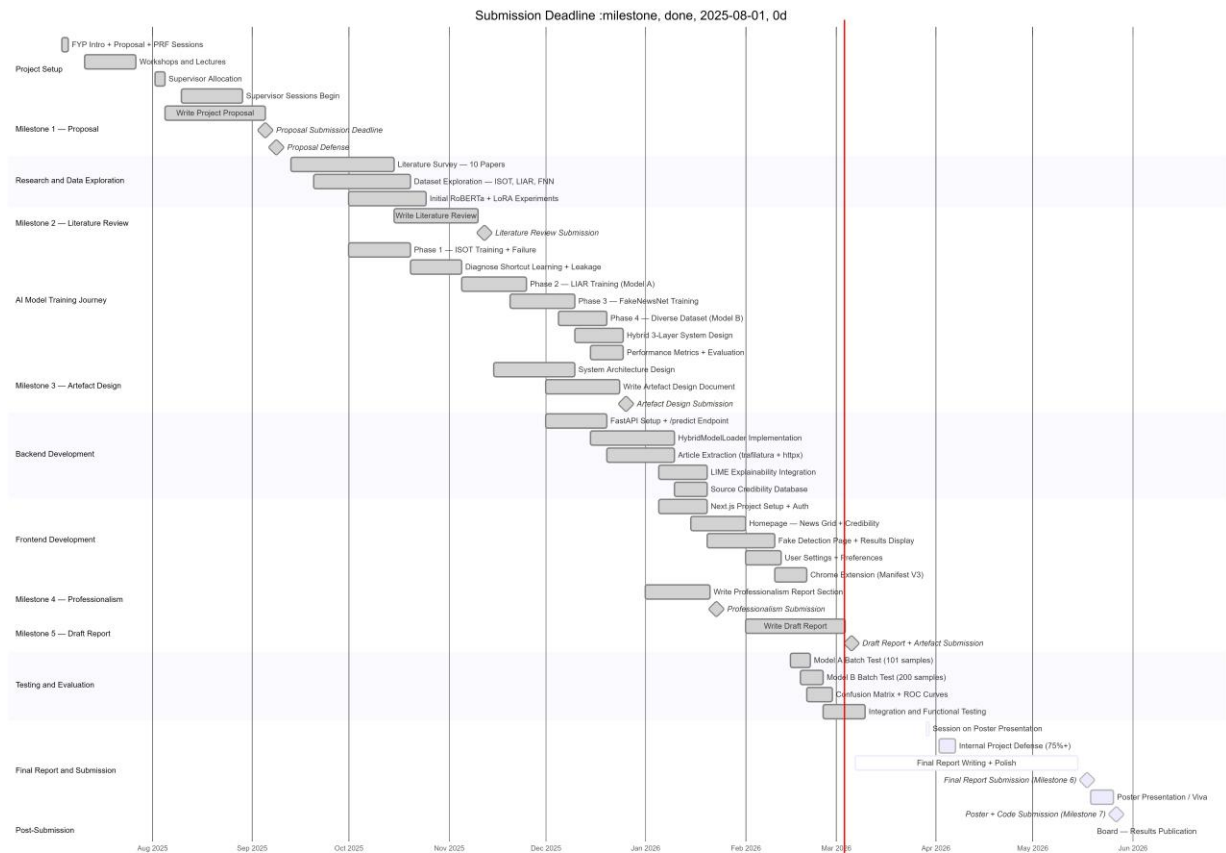


Figure 37: Full Gantt Chart

References and Bibliography

- [1] Abedalla, A., Al-Sadi, A. and Abdullah, M. (2021) 'A Closer Look at Fake News Detection: A Deep Learning Perspective', in Proceedings of the 2021 International Conference on Social Computing. IEEE.
- [2] Mridha, M.F., Keya, A.J., Hamid, M.A., Monowar, M.M. and Rahman, M.S. (2021) 'A Comprehensive Review on Fake News Detection with Deep Learning', IEEE Access, 9, pp. 156151–156170.
- [3] Pavlov, T. and Mirceva, G. (2022) 'COVID-19 Fake News Detection by Using BERT and RoBERTa models', in Proceedings of the Faculty of Computer Science and Engineering, Ss. Cyril and Methodius University.
- [4] Suryavardan, S. et al. (2023) 'Factify 2: A Multimodal Fake News and Satire News Dataset', in Proceedings of the AAAI Workshop on Artificial Intelligence for Social Good.
- [5] Jain, A. (2018) 'Fake News Detection', in IEEE International Students' Conference on Electrical, Electronics and Computer Sciences (SCEECS). IEEE.
- [6] Singhal, S., Dhawan, M., Shah, R.R. and Kumaraguru, P. (2022) 'Inter-modality Discordance for Multimodal Fake News Detection', in Proceedings of the ACM Web Conference 2022.
- [7] Zhang, X., Dadkhah, S., Weismann, A.G., Kanaani, M.A. and Ghorbani, A.A. (2024) 'Multimodal Fake News Analysis Based on Image-Text Similarity', IEEE Transactions on Computational Social Systems, 11(1), pp. 959–972.
- [8] Giachanou, A., Zhang, G. and Rosso, P. (2022) 'Multimodal Multi-image Fake News Detection', in Proceedings of the International Conference on Information Processing and Management.
- [9] Safdar, S. and Wasim, M. (2024) 'RAFAH: RoBERTa Fine-Tuned Model and a Hybrid Features Set for Fake News Detection', in 18th International Conference on Open Source Systems and Technologies (ICOSST). IEEE.

- [10] Gautam, M. and Timilsina, S. (2022) 'NepBERTa: Nepali Language Model Trained in a Large Corpus', in Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the ACL (Volume 2: Short Papers), pp. 273–284.
- [11] Liu, Y. et al. (2019) 'RoBERTa: A Robustly Optimized BERT Pretraining Approach', arXiv preprint arXiv:1907.11692.
- [12] Hu, E.J. et al. (2022) 'LoRA: Low-Rank Adaptation of Large Language Models', in Proceedings of the International Conference on Learning Representations (ICLR).
- [13] Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) "'Why Should I Trust You?': Explaining the Predictions of Any Classifier", in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144.
- [14] Devlin, J. et al. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', in Proceedings of NAACL-HLT 2019.
- [15] Wolf, T. et al. (2020) 'Transformers: State-of-the-Art Natural Language Processing', in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38–45.
- [16] Shu, K. et al. (2020) 'FakeNewsNet: A Data Repository with News Content, Social Context, and Spatiotemporal Information for Studying Fake News on Social Media', Big Data, 8(3), pp. 171–188.
- [17] Wang, W.Y. (2017) "'Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection', in Proceedings of the 55th Annual Meeting of the ACL, pp. 422–426.

Appendices

Appendix A: Mathematical Derivations

A) Self-Attention Formula

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

B) SoftMax Function

$$P(class_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

C) Binary Cross-Entropy Loss

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

D) LoRA Decomposition

$$W' = W + BA$$

Appendix B: User Manual

Web Application:

1. Navigate to the TruthLens homepage to browse curated news with credibility signals.
2. Click 'Fake Detection' in the navigation bar to access the detection tool.
3. Paste article text into the text area and/or enter a source URL.
4. Select the input mode (Auto is recommended for most cases).
5. Click 'Analyse' to submit the text for analysis.
6. Review the verdict (Suspicious/Uncertain/Likely Real), risk level, and step-by-step analysis trace.
7. Click 'Explain' to generate LIME token-level importance highlighting.
8. Create an account via 'Sign Up' to save preferences (news country, categories, default detection settings).

Browser Extension:

1. Open chrome://extensions and enable Developer Mode.
2. Click 'Load unpacked' and select the extension/ folder.
3. Click the TruthLens extension icon while browsing.
4. Paste text or URL into the popup form and click 'Analyze'.
5. Review the verdict and details in the popup result pane.

Appendix C: System Configuration

Backend Setup:

- Python 3.10+ with virtual environment
- Install dependencies: `pip install -r requirements.txt`
- Configure `.env` with `HEADLINE_MODEL_PATH` and `ARTICLE_MODEL_PATH`
- Run: `uvicorn app.main:app --host 0.0.0.0 --port 8000`

Frontend Setup:

- Node.js 18+ with npm
- Install dependencies: `npm install`
- Configure `.env.local` with `BACKEND_URL`, `NEXTAUTH_SECRET`, `MongoDB` connection, and provider keys as needed
- Run: `npm run dev` (development) or `npm run build && npm start` (production)